In [ ]:
```
import base64
import hashlib
import json
from pathlib import Path

import marimo as mo
import pandas as pd

from voc_demo import reader as co
```

Review topics and sentiment · published demo

Loads a pinned workable dataset and S3 demo snapshot so everyone starts from the same results. The Studio *overview* view explains the processing and presents the results. Matching stage caches are reused. LLM calls and demo publishing remain explicit buttons.

Overview

We will begin with an overview of the base system we need to build and describe the processing at each stage. We use the abstraction of a pipeline because it helps explain the data transformations and the algorithms applied at each stage. This is a high-level overview: the components are subject to change and adjustment.



1. Latest release of our Dataset

In [ ]:
```
dataset_refresh = mo.ui.run_button(label="Reload pinned workable dataset")
dataset_refresh
```

Reload pinned workable dataset

In [ ]:
```
_ = dataset_refresh.value
DEMO_VERSION = "188f1d136ccc01f9"
DEMO_RELEASE_ID = "b7a61a8b-27d8-46a7-b2ce-67d97268b741"
snapshot = co.get_dataset(stage="prepared", source="s3", version=DEMO_RELEASE_ID)
workable = snapshot.data
mo.md(
    f"Loaded workable release `{snapshot.info['release_id']}` "
    f"({snapshot.info['schema_version']}, published {snapshot.info['created_at'][:19]}Z) ·
"
```

```
    f"**{len(workable):,}** records × **{workable.shape[1]}** columns"
)
```

Loaded workable release `b7a61a8b-27d8-46a7-b2ce-67d97268b741` (workable-v1, published 2026-09-27T21:52:07Z) · 80,079 records × 18 columns

In [ ]:

```
# Compact, JSON-compatible summary projected into the Studio view as `studio_payload`.
# Counts and dates only; no review text leaves the notebook.
_dates = pd.to_datetime(workable["record_date"], errors="coerce", utc=True,
format="mixed")
_platform_order = ["google_play", "app_store", "x", "youtube"]
_present = [p for p in _platform_order if (workable["platform"] == p).any()]
_present += sorted(set(workable["platform"].dropna().unique()) - set(_present))


def _earliest(series):
    return series.min().strftime("%Y-%m-%d") if series.notna().any() else None


def _latest(series):
    return series.max().strftime("%Y-%m-%d") if series.notna().any() else None


_platforms = []
for _name in _present:
    _rows = workable["platform"].eq(_name)
    _d = _dates[_rows]
    _rating = workable.loc[_rows, "rating"].dropna()
    _platforms.append(
        {
            "platform": _name,
            "records": int(_rows.sum()),
            "share": round(float(_rows.mean()), 4),
            "first_record": _earliest(_d),
            "last_record": _latest(_d),
            "providers": sorted(workable.loc[_rows,
"preferred_provider"].dropna().unique().tolist()),
            "rated": int(_rating.size),
            "mean_rating": round(float(_rating.mean()), 2) if _rating.size else None,
        }
    )

_monthly = (
    workable.assign(month=_dates.dt.strftime("%Y-%m"))
    .pivot_table(index="month", columns="platform", values="record_id", aggfunc="count",
fill_value=0)
    .sort_index()
)
_months = [
    {"month": str(_month), **{str(_p): int(_n) for _p, _n in _row.items()}}
    for _month, _row in _monthly.iterrows()
]

_info = snapshot.info
studio_payload = {
```

```
    "release": {
        key: _info.get(key)
        for key in (
            "release_id", "created_at", "publisher_id", "schema_version", "stage",
"source",
            "artifact_id", "raw_revision_id", "row_count", "size_bytes",
        )
    },
    "loaded_at": pd.Timestamp.now(tz="UTC").isoformat(timespec="seconds"),
    "counts": {
        "rows": int(len(workable)),
        "columns": int(workable.shape[1]),
        "platforms": len(_platforms),
        "unknown_dates": int(_dates.isna().sum()),
    },
    "dates": {"first_record": _earliest(_dates), "last_record": _latest(_dates)},
    "platforms": _platforms,
    "months": _months,
    "providers": [{"provider": str(k), "records": int(v)} for k, v in
workable["preferred_provider"].value_counts().items()],
    "relevance": [{"relevance": str(k), "records": int(v)} for k, v in
workable["relevance"].value_counts().items()],
    "columns": [
        {"name": str(c), "dtype": str(workable[c].dtype), "non_null":
int(workable[c].notna().sum())}
        for c in workable.columns
    ],
}
mo.md(
    f"`studio_payload` ready for the Studio *overview* view · {studio_payload['counts']
['rows']:,} records, "
    f"{studio_payload['counts']['platforms']} platforms, {len(_months)} months"
)
```

`studio_payload` ready for the Studio *overview* view · 80,079 records, 4 platforms, 13 months

2. Embeddings

Extract 100 samples

In [ ]:
```
from voc_demo.settings import data_root
from voc_demo.checkpoints import seed_prototype_cache

SAMPLE_SEED = 42
SAMPLE_DIR = data_root() / "workflow01" / DEMO_VERSION
demo_seed = seed_prototype_cache(
    SAMPLE_DIR, release_id=snapshot.info["release_id"],
    workable=workable, version=DEMO_VERSION,
)
embeddings_sample = pd.read_parquet(SAMPLE_DIR / "sample.parquet")
_origin = "reused the published sample; no replacement sample is generated"
_mix = ", ".join(f"{k} {v:,}" for k, v in
embeddings_sample["platform"].value_counts().items())
mo.vstack(
```

```
    [
        mo.md(
            f"**{len(embeddings_sample):,}** records, {_origin} · platform mix: {_mix}"
            + f" · {demo_seed['source']}"
            + (f" · {demo_seed['message']}" if demo_seed["message"] else "")
        ),
        mo.ui.table(
            embeddings_sample[["record_id", "platform", "record_date", "rating", "text"]],
            label="Embeddings sample",
            page_size=10,
        ),
    ]
)
```

1,000 records, reused the published sample; no replacement sample is generated · platform mix: google_play 837, x 87, app_store 67, youtube 9 · existing cache · Existing stage files preserved

Embeddings sample

Search...

record_id string unique: 1,000	platform string unique: 4	record_date string unique: 991	rating Int64 (nulls: 96)	text string unique: 838
x:2020914338698011000	x	2026-02-09T17:34:59+00:00	None	@http_HectorLN ¡Hola, Héctor! Lamentar
play:com.citibanamex.banamexmobile:85203f...	google_play	2026-08-13T14:26:32+00:00	1	Solo me pase con ustedes por el token fís
appbot:498483038:3640102715	app_store	2025-11-20	1	No aparecen los movimientos de queda c
play:com.citibanamex.banamexmobile:91f0de...	google_play	2025-09-30T18:18:20+00:00	5	fantástica
play:com.citibanamex.banamexmobile:ea130e...	google_play	2026-08-30T19:47:57+00:00	5	exelente
play:com.citibanamex.banamexmobile:b37713...	google_play	2026-02-11T22:48:57+00:00	5	Siempre disponible y super fácil de usar
play:com.citibanamex.banamexmobile:d0a671...	google_play	2025-10-30T17:59:08+00:00	5	excelente atención
x:2007157160669597724	x	2026-01-02T18:28:52+00:00	None	Nuestros cachorritos están en busca de p
play:com.citibanamex.banamexmobile:3b1adc...	google_play	2026-01-18T14:35:13+00:00	5	excelente
play:com.citibanamex.banamexmobile:eebfa5...	google_play	2026-05-28T17:49:39+00:00	5	Muy buena aplicación.

1,000 rows, 5 columns — *No selection*

Prepare for embedding

The encoder receives plain text, one string per record, so preparation is bookkeeping rather than modelling:

1. Keep records whose `model_text` is not blank after stripping surrounding whitespace.
2. Copy that text into `embedding_text`, the exact input the encoder will see.
3. Reuse `text_hash` as `input_hash`: in the workable release it is already the SHA-256 of this text, so every vector can be traced back to its input.
4. Collect the inputs in the list `texts`. Row order is the link: `prepared_sample.iloc[i]` ↔ `texts[i]` ↔ `embeddings[i]`.

Exact duplicate texts are kept so the rows stay aligned with the sample; the embedding step can encode unique texts once and reuse them.

In [ ]:
```
# Preparation only: plain text plus bookkeeping. No vectors are produced here.
_text = embeddings_sample["model_text"].fillna("").str.strip()
```

```python
prepared_sample = embeddings_sample.loc[_text.ne("")].copy()
prepared_sample["embedding_text"] = _text.loc[prepared_sample.index]
# text_hash in the workable release is already the SHA-256 of this exact text.
prepared_sample["input_hash"] = prepared_sample["text_hash"]
prepared_sample = prepared_sample.reset_index(drop=True)

texts = prepared_sample["embedding_text"].tolist()  # texts[i] <-> prepared_sample.iloc[i]
<-> embeddings[i]
assert all(isinstance(_t, str) and _t.strip() for _t in texts)

_lengths = prepared_sample["embedding_text"].str.len()
_duplicates = int(prepared_sample["input_hash"].duplicated().sum())
mo.vstack(
    [
        mo.md(
            f"**{len(texts):,}** texts ready ({len(embeddings_sample) - len(texts)} blank
dropped) · "
            f"{_duplicates} exact duplicates kept for row alignment · "
            f"length: median {int(_lengths.median())} chars, max {int(_lengths.max())}"
        ),
        mo.ui.table(
            prepared_sample[["record_id", "platform", "embedding_text", "input_hash"]],
            label="Prepared inputs",
            page_size=10,
        ),
    ]
)
```

1,000 texts ready (0 blank dropped) · 162 exact duplicates kept for row alignment · length: median 35 chars, max 733

Prepared inputs

Search...

record_id string unique: 1,000	platform string unique: 4	embedding_text string unique: 838	input_hash string unique: 838
x:2020914338698011000	x	@http_HectorLN ¡Hola, Héctor! Lamentamos ...	711b9f8e30625df7dfdcd6535993667f
play:com.citibanamex.banamexmobile:85203f...	google_play	Solo me pase con ustedes por el token físico p...	3dec558461f3f91b85793cc1d3e0993e
appbot:498483038:3640102715	app_store	No aparecen los movimientos de queda carga...	05ccbf2f8fb8e12e8b977243328ef022
play:com.citibanamex.banamexmobile:91f0de...	google_play	fantástica	4f913484d7f0d6b2316314730bcd8aa
play:com.citibanamex.banamexmobile:ea130e...	google_play	exelente	510beb86bf6885d26999dc80a3741ab
play:com.citibanamex.banamexmobile:b37713...	google_play	Siempre disponible y super fácil de usar	c680df408d44012e2f80088a7a7c1ec3
play:com.citibanamex.banamexmobile:d0a671...	google_play	excelente atención	5e6a837b46e145c4f1e761016be0f1bc
x:2007157160669597724	x	Nuestros cachorritos están en busca de padrin...	30af199bc7743086b4f82659cfcc27d8
play:com.citibanamex.banamexmobile:3b1adc...	google_play	excelente	ab06f640e033b435c72b3dfd292c20de
play:com.citibanamex.banamexmobile:eebfa5...	google_play	Muy buena aplicación.	1430a0a780aac714ea7d1c9ec35e581

1,000 rows, 4 columns — *No selection*

Embed

We use `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`, the encoder evaluated in `embeddings-es.py`, which handles Spanish reviews well: it maps each text to a 384-dimensional vector where paraphrases land close together. The revision is pinned so the vectors are reproducible.

Two details matter for later stages:

- Vectors are L2-normalised, so cosine similarity is a plain dot product and the distance-based steps (dimensionality reduction, clustering) behave consistently.
- The encoder reads at most 128 tokens; longer reviews are truncated, so we count them.

The result is a separate matrix `embeddings` with one row per prepared text, aligned with `prepared_sample` and `texts`. The text columns are never replaced. The sample and its vectors are saved under `.data/workflow01/<snapshot-id>/`; later runs reuse them when the inputs and model match.

In [ ]:
```
from voc_demo.settings import model_root as _model_root
from sentence_transformers import SentenceTransformer
import numpy as np

MODEL_ID = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
MODEL_REVISION = "e8f8c211226b894fcb81acc59f3b34ba3efd5f42"  # pinned for reproducible
vectors

encoder = SentenceTransformer(
    MODEL_ID,
    revision=MODEL_REVISION,
    cache_folder=str(_model_root() / "sentence-transformers"),
    device="cpu",
)

# Token lengths without truncation, to see how many inputs the encoder will cut.
_tokens = encoder.tokenizer(texts, truncation=False, padding=False,
add_special_tokens=True, verbose=False)
token_lengths = np.array([len(_ids) for _ids in _tokens["input_ids"]])
_truncated = int((token_lengths > encoder.max_seq_length).sum())
mo.md(
    f"Encoder `{MODEL_ID}` @ `{MODEL_REVISION[:12]}` · **
{encoder.get_sentence_embedding_dimension()}** dimensions · "
    f"token limit **{encoder.max_seq_length}** · inputs: median
{int(np.median(token_lengths))} tokens, "
    f"max {int(token_lengths.max())}, **{_truncated}** will be truncated"
)
```

```
Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]
Loading weights:  55%|█████▌    | 109/199 [00:00<00:00, 955.14it/s]
Loading weights: 100%|██████████| 199/199 [00:00<00:00, 1310.89it/s]
/tmp/marimo_196729/__marimo__cell_nWHF_.py:20: FutureWarning: The `get_sentence_embedding_di
mension` method has been renamed to `get_embedding_dimension`.
  f"Encoder `{MODEL_ID}` @ `{MODEL_REVISION[:12]}` · **{encoder.get_sentence_embedding_dimens
ion()}** dimensions · "
```

Encoder `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2` @ `e8f8c211226b` · 384 dimensions · token limit 128 · inputs: median 10 tokens, max 170, 4 will be truncated

In [ ]:
```
# Reuse saved vectors when they were produced by this model for exactly these inputs;
otherwise encode and save.
_vectors_path = SAMPLE_DIR / "embeddings.npy"
_manifest_path = SAMPLE_DIR / "embeddings.json"
```

```
_expected = {
    "model": MODEL_ID,
    "revision": MODEL_REVISION,
    "normalized": True,
    "rows": len(texts),
    "inputs_sha256": hashlib.sha256("".join(prepared_sample["input_hash"]).encode("utf-
8")).hexdigest(),
}
_saved = json.loads(_manifest_path.read_text(encoding="utf-8")) if _manifest_path.exists()
else None
if _saved == _expected and _vectors_path.exists():
    embeddings = np.load(_vectors_path, allow_pickle=False)
    _origin = f"loaded from `{_vectors_path.name}`"
else:
    embeddings = encoder.encode(
        texts,
        batch_size=64,
        show_progress_bar=False,
        normalize_embeddings=True,
        convert_to_numpy=True,
    )
    np.save(_vectors_path, embeddings, allow_pickle=False)
    _manifest_path.write_text(json.dumps(_expected, indent=2), encoding="utf-8")
    _origin = f"encoded now and saved to `{_vectors_path.name}`"
assert embeddings.shape == (len(prepared_sample),
encoder.get_sentence_embedding_dimension())
assert np.isfinite(embeddings).all()
assert np.allclose(np.linalg.norm(embeddings, axis=1), 1, atol=1e-5)

_preview = prepared_sample[["record_id", "platform", "embedding_text"]].head(5).copy()
for _d in range(5):
    _preview[f"dim_{_d}"] = embeddings[:5, _d].round(4)
mo.vstack(
    [
        mo.md(
            f"`embeddings` is a **{embeddings.shape[0]:,} × {embeddings.shape[1]}**
float32 matrix, {_origin}. "
            f"Unit-norm rows, aligned with `prepared_sample` (row *i* ↔ `texts[i]`)."
        ),
        mo.ui.table(_preview, label="Embedding preview: first five records, dimensions 0-4
of 384"),
    ]
)
```

```
/tmp/marimo_196729/__marimo__cell_iLit_.py:26: FutureWarning: The `get_sentence_embedding_di
mension` method has been renamed to `get_embedding_dimension`.
  assert embeddings.shape == (len(prepared_sample), encoder.get_sentence_embedding_dimension
())
```

`embeddings` is a **1,000 × 384** float32 matrix, loaded from `embeddings.npy`. Unit-norm rows, aligned with `prepared_sample` (row *i* ↔ `texts[i]`).

Embedding preview: first five records, dimensions 0–4 of 384

Search...

record_id string	platform string	embedding_text string	dim_0 float32	dim_1 float32
x:2020914338698011000	x	@http_HectorLN ¡Hola, Héctor! Lamentamos ...	-0.103399999439716340	0.026799
play:com.citibanamex.banamexmobile:85203f...	google_play	Solo me pase con ustedes por el token físico p...	-0.042199999094009400	-0.023299
appbot:498483038:3640102715	app_store	No aparecen los movimientos de queda carga...	-0.020600000396370888	-0.016499
play:com.citibanamex.banamexmobile:91f0de...	google_play	fantástica	0.028000000864267350	0.010300
play:com.citibanamex.banamexmobile:ea130e...	google_play	exelente	0.041600000113248825	0.059999

5 rows, 8 columns *No selection*

Nearest neighbours

Pick a review and the five reviews whose vectors are closest to it appear below. Because the vectors are unit-norm, the cosine similarity is a dot product: `embeddings @ embeddings[i]`. Neighbours are found with all 384 coordinates; this is the raw signal the next stages work with.

In [ ]:
```
review_position = mo.ui.slider(
    start=0, stop=len(prepared_sample) - 1, step=1, value=0, label="Review position",
show_value=True
)
review_position
```

Review position 0

In [ ]:
```
_i = int(review_position.value)
_scores = embeddings @ embeddings[_i]
_order = np.argsort(-_scores)
_neighbors = _order[_order != _i][:5]  # exclude the review itself
_matches = prepared_sample.iloc[_neighbors][["record_id", "platform",
"embedding_text"]].copy()
_matches.insert(0, "cosine_similarity", _scores[_neighbors].round(3))
mo.vstack(
    [
        mo.md("**Selected review**"),
        mo.ui.table(prepared_sample.iloc[[_i]][["record_id", "platform",
"embedding_text"]]),
        mo.md("**Five nearest reviews in this sample**"),
        mo.ui.table(_matches),
    ]
)
```

Selected review

Search...

	Index0 int64	record_id string	platform string	embedding_text string
☐	0	x:2020914338698011000	x	@http_HectorLN ¡Hola, Héctor! Lamentamos que no pued...

1 row, 3 columns — *No selection*

Five nearest reviews in this sample

Search...

	Index0 int64	cosine_similarity float32	record_id string	platform string	embedding_text string
☐	250	0.8109999895095825	x:2024565249743950279	x	@starmetaphors ¡Hola, @starmetaphors! L
☐	210	0.7680000066757202	x:2026025111875461317	x	@OscrC272668z ¡Hola, @OscrC272668z! S
☐	314	0.6899999976158142	play:com.citibanamex.banamexmobile:d7e046...	google_play	muchísimas gracias por la atención informa
☐	431	0.6589999794960022	play:com.citibanamex.banamexmobile:d17776...	google_play	excelente servicio de Banamex 👏👏👏👏
☐	515	0.6179999709129333	x:2052812130152284368	x	@CondusefMX @almaphantom Recomiend

5 rows, 4 columns — *No selection*

In [ ]:

```
# Projection for the Studio "Embeddings" page: model facts, the sampled texts, exact top-5 neighbours
# for every row, and an int8-quantised copy of the vectors (base64) so the page stays under Studio's
# 1 MB value limit. Exact values for the first eight dimensions travel separately.
_sims = embeddings @ embeddings.T
np.fill_diagonal(_sims, -np.inf)
_top = np.argsort(-_sims, axis=1)[:, :5]
_max_abs = float(np.abs(embeddings).max())
_quantised = np.clip(np.round(embeddings / _max_abs * 127), -127, 127).astype(np.int8)
_rows = prepared_sample[["record_id", "platform", "embedding_text"]]
embedding_payload = {
    "model": {
        "id": MODEL_ID,
        "revision": MODEL_REVISION,
        "dimensions": int(embeddings.shape[1]),
        "max_tokens": int(encoder.max_seq_length),
        "normalized": True,
    },
    "stats": {
        "rows": int(len(texts)),
        "median_tokens": int(np.median(token_lengths)),
        "max_tokens_seen": int(token_lengths.max()),
        "truncated": int((token_lengths > encoder.max_seq_length).sum()),
        "duplicate_texts": int(prepared_sample["input_hash"].duplicated().sum()),
        "sample_seed": SAMPLE_SEED,
    },
    "rows": [
        {"record_id": str(r.record_id), "platform": str(r.platform), "text":
str(r.embedding_text), "tokens": int(t)}
        for r, t in zip(_rows.itertuples(index=False), token_lengths)
    ],
    "neighbors": [[[int(j), round(float(_sims[i, j]), 4)] for j in _top[i]] for i in
range(len(_top))],
    "heads": [[round(float(x), 4) for x in row[:8]] for row in embeddings],
    "vectors": {
```

```
        "dims": int(embeddings.shape[1]),
        "scale": _max_abs / 127,
        "int8_base64": base64.b64encode(_quantised.tobytes()).decode("ascii"),
    },
}
_size = len(json.dumps(embedding_payload, ensure_ascii=False).encode("utf-8"))
assert _size < 950_000, f"embedding_payload is {_size:,} bytes; Studio projections must
stay under 1 MB"
mo.md(f"`embedding_payload` ready for the Studio *Embeddings* page · {_size / 1000:,.0f}
kB · max |value| {_max_abs:.3f}")
```

`embedding_payload` ready for the Studio *Embeddings* page · 861 kB · max |value| 0.557

3. Dimensionality reduction

The problem: we can measure closeness, but we cannot see it

We now have a way of knowing how close two reviews are: the cosine similarity of their vectors. The next idea follows naturally. If we can find groups of vectors that sit close together, those groups should hold reviews about similar subjects, and those subjects are what we want to explore.

The obstacle is that every vector has 384 dimensions, and we can only picture two or three at a time. The intuition is easy to check: pick any two dimensions and a pair of reviews may look close; pick two other dimensions and the same pair looks far apart. No single pair of axes tells the truth, because the distance lives in all 384 coordinates at once. Clustering algorithms meet the same problem in another form: with so many dimensions, distances between points become more alike and groups harder to separate.

This is where dimensionality reduction helps: it builds a few new axes that keep as much of the neighbourhood structure as possible, so that *close* means close wherever you look.

In [ ]:
```
# Two reviews that are genuinely close in 384-D but written differently: among pairs with
cosine
# similarity ≥ 0.75 and different wording, the pair whose two most divergent coordinates
are largest,
# measured in units of each dimension's spread across the sample.
_pair_sims = embeddings @ embeddings.T
np.fill_diagonal(_pair_sims, -1.0)
_wording = (
    prepared_sample["embedding_text"].str.lower().str.replace(r"\W+", " ",
regex=True).str.strip().to_numpy()
)
_spread = embeddings.std(axis=0)
_rows, _cols = np.where(np.triu((_pair_sims >= 0.75) & (_wording[:, None] !=
_wording[None, :]), 1))
_best = None
for _i, _j in zip(_rows, _cols):
    _gap = np.abs(embeddings[_i] - embeddings[_j]) / _spread
    _top = np.argsort(-_gap)[:2]
```

```
    _score = float(np.hypot(_gap[_top[0]], _gap[_top[1]]))
    if _best is None or _score > _best[0]:
        _best = (_score, int(_i), int(_j), _gap)
_, pair_a, pair_b, _gap = _best
pair_similarity = float(_pair_sims[pair_a, pair_b])
close_dims = tuple(int(d) for d in np.argsort(_gap)[:2])  # coordinates where the two
almost coincide
far_dims = tuple(int(d) for d in np.argsort(-_gap)[:2])  # coordinates where they differ
most
mo.md(
    f"**Review A** (row {pair_a}, {prepared_sample.loc[pair_a, 'platform']}): "
    f"“{prepared_sample.loc[pair_a, 'embedding_text']}”\n\n"
    f"**Review B** (row {pair_b}, {prepared_sample.loc[pair_b, 'platform']}): "
    f"“{prepared_sample.loc[pair_b, 'embedding_text']}”\n\n"
    f"Cosine similarity across all 384 dimensions: **{pair_similarity:.3f}**, among the
closest pairs in the sample. "
    f"Yet in dimensions {far_dims} they sit far apart, while in dimensions {close_dims}
they almost coincide."
)
```

Review A (row 88, app_store): “Rápida, eficiente, práctica, segura!!” Review B (row 501, google_play): “muy eficiente” Cosine similarity across all 384 dimensions: 0.768, among the closest pairs in the sample. Yet in dimensions (351, 233) they sit far apart, while in dimensions (190, 132) they almost coincide.

In [ ]:
```
dim_x = mo.ui.slider(start=0, stop=embeddings.shape[1] - 1, step=1, value=far_dims[0],
label="x axis: dimension", show_value=True)
dim_y = mo.ui.slider(start=0, stop=embeddings.shape[1] - 1, step=1, value=far_dims[1],
label="y axis: dimension", show_value=True)
mo.hstack([dim_x, dim_y], justify="start", gap=2)
```

x axis: dimension 351 y axis: dimension 233

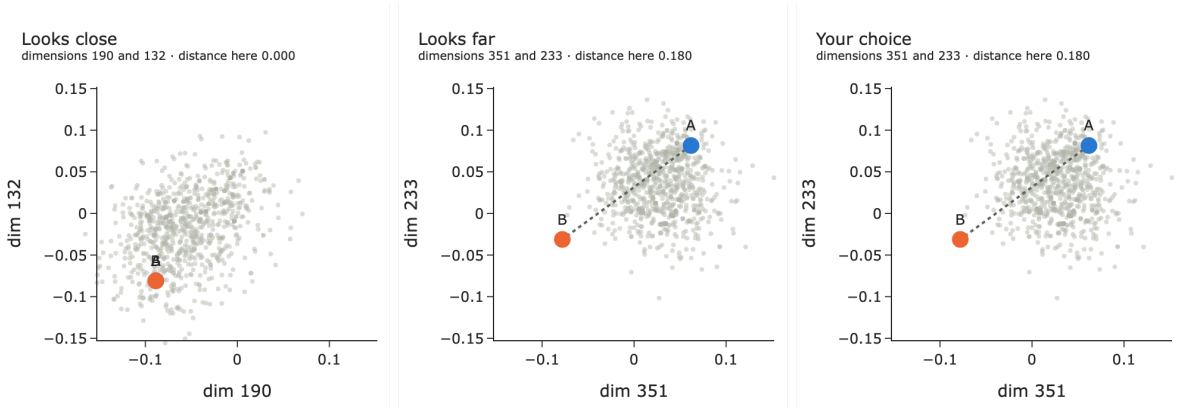
In [ ]:
```
import plotly.graph_objects as go

_limit = float(np.percentile(np.abs(embeddings), 99.5))  # one shared axis range, so
distances compare across panels


def _panel(dims, title):
    _x, _y = dims
    _ax, _ay = embeddings[pair_a, _x], embeddings[pair_a, _y]
    _bx, _by = embeddings[pair_b, _x], embeddings[pair_b, _y]
    _figure = go.Figure()
    _figure.add_scatter(
        x=embeddings[:, _x], y=embeddings[:, _y], mode="markers", name="other reviews",
        marker={"size": 4, "color": "#b4b7ac", "opacity": 0.45}, hoverinfo="skip",
    )
    _figure.add_scatter(x=[_ax, _bx], y=[_ay, _by], mode="lines", line={"color":
"#60645a", "dash": "dot"}, showlegend=False, hoverinfo="skip")
    _figure.add_scatter(x=[_ax], y=[_ay], mode="markers+text", name="Review A", text=
["A"], textposition="top center", marker={"size": 14, "color": "#2a78d6"})
    _figure.add_scatter(x=[_bx], y=[_by], mode="markers+text", name="Review B", text=
["B"], textposition="top center", marker={"size": 14, "color": "#eb6834"})
```

```
    _distance = float(np.hypot(_ax - _bx, _ay - _by))
    _figure.update_layout(
        title={"text": f"{title}<br><sup>dimensions {_x} and {_y} · distance here
{_distance:.3f}</sup>", "font": {"size": 14}},
        xaxis={"title": f"dim {_x}", "range": [-_limit, _limit]}, yaxis={"title": f"dim
{_y}", "range": [-_limit, _limit]},
        height=340, margin={"l": 50, "r": 10, "t": 70, "b": 45}, showlegend=False,
template="simple_white",
    )
    return mo.ui.plotly(_figure)


mo.vstack(
    [
        mo.hstack(
            [_panel(close_dims, "Looks close"), _panel(far_dims, "Looks far"),
_panel((dim_x.value, dim_y.value), "Your choice")],
            widths="equal",
        ),
        mo.md(
            "Same two reviews, same similarity, three different pictures. Two coordinates
out of 384 are not evidence of closeness; "
            "move the sliders to look through any other pair. Dimensionality reduction
builds axes in which the first picture is the honest one."
        ),
    ]
)
```



Same two reviews, same similarity, three different pictures. Two coordinates out of 384 are not evidence of closeness; move the sliders to look through any other pair. Dimensionality reduction builds axes in which the first picture is the honest one.

What the book says

Hands-On Large Language Models, ch. 5: as the number of dimensions grows, the space a clustering algorithm has to search grows exponentially and distances become less informative, so clustering 384-dimensional vectors directly tends to find poor groups. The second step of the pipeline is therefore to *compress* the embeddings into a few dimensions while keeping their global structure. The book picks UMAP over PCA because it handles non-linear structure better, and uses `n_components=5` (5–10 "work well"), `min_dist=0.0` for tighter groups, `metric="cosine"` because Euclidean distances misbehave in high

dimensions, and a `random_state` for reproducibility (which disables parallelism). It warns that every reduction loses information, and that a separate 2-D reduction is only for looking at the data: it can push groups together or apart, so human inspection stays essential.

What UMAP is

Uniform Manifold Approximation and Projection (McInnes, Healy & Melville, 2018). It first builds a weighted graph in the original space: each review is linked to its `n_neighbors` nearest reviews (by cosine distance here), with weights that fall off with distance. It then places every point in the low-dimensional space and moves them with gradient descent until the neighbour graph of the new positions matches the original graph as closely as possible, pulling neighbours together and pushing non-neighbours apart. Intuition: *keep each review's neighbours as its neighbours*. `n_neighbors` trades local detail for global shape; `min_dist` sets how tightly neighbours may pack.

In our flow

From the same saved vectors we fit two reductions: 5-D (`reduced_embeddings`), the input of the clustering stage, and 2-D (`coordinates_2d`), used only to draw the sample. Both are saved beside the embeddings and reused while the vectors and parameters are unchanged. A small check below measures how much neighbourhood structure survives.

In [ ]:
```
import time
from importlib import metadata as importlib_metadata

from umap import UMAP

UMAP_NEIGHBORS = 15  # size of each point's neighbourhood in the original space (UMAP's default)
CLUSTER_DIMENSIONS = 5  # the book's choice for the clustering input; 5-10 keeps global structure
UMAP_SEED = 42  # reproducible fits, single-threaded

_reduced_path = SAMPLE_DIR / "reduced.npz"
_reduced_manifest = SAMPLE_DIR / "reduced.json"
_expected_reduction = {
    "embeddings_sha256": hashlib.sha256(embeddings.tobytes()).hexdigest(),
    "n_neighbors": UMAP_NEIGHBORS,
    "cluster_dimensions": CLUSTER_DIMENSIONS,
    "min_dist": {"cluster": 0.0, "display": 0.1},
    "metric": "cosine",
    "seed": UMAP_SEED,
    "umap_learn": importlib_metadata.version("umap-learn"),
}
_saved_reduction = (
    json.loads(_reduced_manifest.read_text(encoding="utf-8")) if
_reduced_manifest.exists() else None
)
if _saved_reduction == _expected_reduction and _reduced_path.exists():
    with np.load(_reduced_path) as _saved:
```

```
        reduced_embeddings, coordinates_2d = _saved["cluster_5d"], _saved["display_2d"]
    _origin = f"loaded from `{_reduced_path.name}`"
else:
    _t0 = time.perf_counter()
    reduced_embeddings = UMAP(
        n_neighbors=UMAP_NEIGHBORS, n_components=CLUSTER_DIMENSIONS, min_dist=0.0,
        metric="cosine", random_state=UMAP_SEED, n_jobs=1,
    ).fit_transform(embeddings)
    coordinates_2d = UMAP(
        n_neighbors=UMAP_NEIGHBORS, n_components=2, min_dist=0.1,
        metric="cosine", random_state=UMAP_SEED, n_jobs=1,
    ).fit_transform(embeddings)
    np.savez(_reduced_path, cluster_5d=reduced_embeddings, display_2d=coordinates_2d)
    _reduced_manifest.write_text(json.dumps(_expected_reduction, indent=2), encoding="utf-
8")
    _origin = f"fitted now in {time.perf_counter() - _t0:.1f} s and saved to
`{_reduced_path.name}`"
assert reduced_embeddings.shape == (len(prepared_sample), CLUSTER_DIMENSIONS)
assert coordinates_2d.shape == (len(prepared_sample), 2)
assert np.isfinite(reduced_embeddings).all() and np.isfinite(coordinates_2d).all()
mo.md(
    f"`reduced_embeddings`: **{embeddings.shape[1]} → {CLUSTER_DIMENSIONS}** dimensions
for clustering · "
    f"`coordinates_2d`: **→ 2** for display · {_origin}. Rows stay aligned with
`prepared_sample`."
)
```

`reduced_embeddings`: 384 → 5 dimensions for clustering · `coordinates_2d`: → 2 for display · loaded from `reduced.npz`. Rows stay aligned with `prepared_sample`.

In [ ]:

```
# How much neighbourhood structure survives the reduction? For every review, compare its
five nearest
# neighbours in the original 384-D space (cosine) with its five nearest in each reduced
space (Euclidean).
from scipy.spatial.distance import cdist


def _top5(distances):
    np.fill_diagonal(distances, np.inf)
    return np.argsort(distances, axis=1)[:, :5]


def _overlap(reduced):
    _original = _top5(1 - embeddings @ embeddings.T)
    _after = _top5(cdist(reduced, reduced))
    return float(np.mean([len(set(a) & set(b)) / 5 for a, b in zip(_original, _after)]))


neighbourhood_overlap = {
    f"{CLUSTER_DIMENSIONS}-D (clustering)": round(_overlap(reduced_embeddings), 3),
    "2-D (display)": round(_overlap(coordinates_2d), 3),
}
mo.md(
    "**Neighbourhood preservation**, the share of each review's five nearest neighbours
that are still "
```

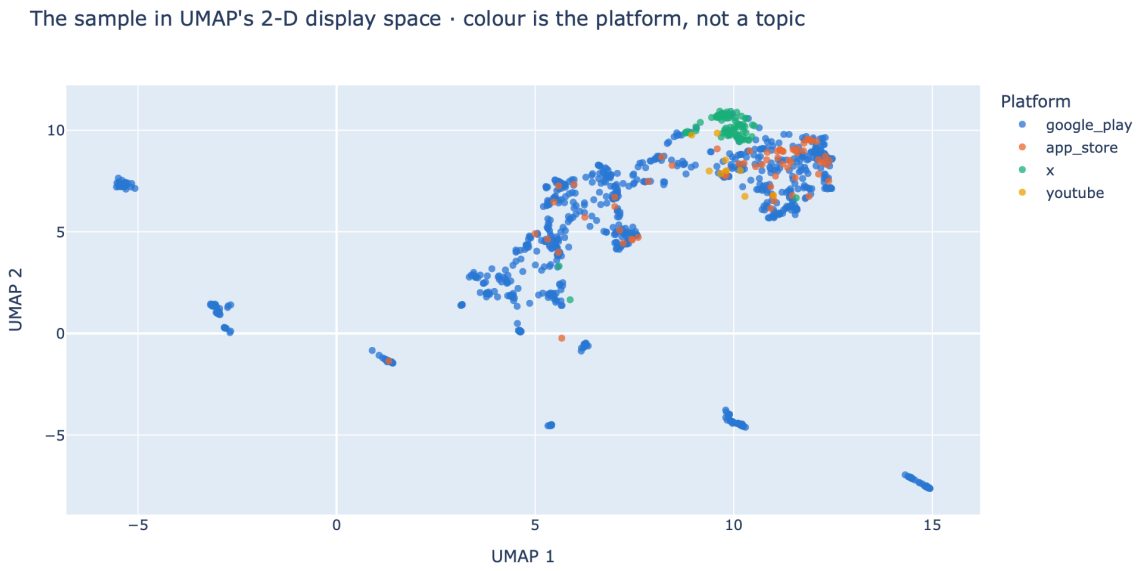
```
    "among its five nearest after reduction: "
    + " · ".join(f"{k}: **{v:.0%}**" for k, v in neighbourhood_overlap.items())
    + "\n\nInformation is lost by design: the reduced space keeps the broad structure that
clustering needs, "
    "not every local detail. Use the 2-D picture to orient yourself, not to decide whether
two reviews are truly close."
)
```

Neighbourhood preservation, the share of each review's five nearest neighbours that are still among its five nearest after reduction: 5-D (clustering): 41% · 2-D (display): 36% Information is lost by design: the reduced space keeps the broad structure that clustering needs, not every local detail. Use the 2-D picture to orient yourself, not to decide whether two reviews are truly close.

In [ ]:

```
import plotly.express as px

PLATFORM_COLOURS = {"google_play": "#2a78d6", "app_store": "#eb6834", "x": "#1baf7a",
"youtube": "#eda100"}
plot_2d = prepared_sample[["record_id", "platform", "embedding_text"]].assign(
    x=coordinates_2d[:, 0],
    y=coordinates_2d[:, 1],
    preview=prepared_sample["embedding_text"].str.slice(0, 90)
    + np.where(prepared_sample["embedding_text"].str.len() > 90, "…", ""),
)
_figure = px.scatter(
    plot_2d,
    x="x",
    y="y",
    color="platform",
    color_discrete_map=PLATFORM_COLOURS,
    category_orders={"platform": list(PLATFORM_COLOURS)},
    hover_name="preview",
    hover_data={"x": False, "y": False, "platform": True, "record_id": True},
    opacity=0.75,
    title="The sample in UMAP's 2-D display space · colour is the platform, not a topic",
)
_figure.update_traces(marker={"size": 6})
_figure.update_layout(height=540, legend_title_text="Platform", xaxis_title="UMAP 1",
yaxis_title="UMAP 2")
mo.ui.plotly(_figure)
```



In [ ]:

```
# Projection for the Studio "Dimensionality reduction" page: the demo pair, UMAP's 2-D coordinates,
# parameters and the neighbourhood-preservation scores. Vectors themselves come from `embedding_payload`.
reduction_payload = {
    "params": {
        "n_neighbors": UMAP_NEIGHBORS,
        "cluster_dimensions": CLUSTER_DIMENSIONS,
        "min_dist_cluster": 0.0,
        "min_dist_display": 0.1,
        "metric": "cosine",
        "seed": UMAP_SEED,
        "umap_learn": importlib_metadata.version("umap-learn"),
    },
    "overlap": {"cluster": neighbourhood_overlap[f"{CLUSTER_DIMENSIONS}-D (clustering)"],
"display": neighbourhood_overlap["2-D (display)"]},
    "pair": {
        "a": pair_a,
        "b": pair_b,
        "similarity": round(pair_similarity, 4),
        "close_dims": list(close_dims),
        "far_dims": list(far_dims),
    },
    "spread": [round(float(s), 5) for s in embeddings.std(axis=0)],
    "axis_limit": round(float(np.percentile(np.abs(embeddings), 99.5)), 4),
    "coordinates_2d": [[round(float(x), 3), round(float(y), 3)] for x, y in
coordinates_2d],
    "input_dimensions": int(embeddings.shape[1]),
}
mo.md(
    f"`reduction_payload` ready for the Studio *Dimensionality reduction* page · "
    f"{len(json.dumps(reduction_payload).encode()) / 1000:,.0f} kB"
)
```

`reduction_payload` ready for the Studio *Dimensionality reduction* page · 20 kB

4. Clustering

Groups in the small space

Each review is now a point in five dimensions whose distances mean something, so we can look for groups: regions where points are packed more tightly than their surroundings. A tight group of vectors is a set of reviews that say similar things, so every group is a *candidate subject*. A group gets a number, not a name; naming it is the next stage.

What the book says

Step three clusters the reduced embeddings (*Hands-On Large Language Models*, ch. 5). The common choice, k-means, needs the number of groups in advance, and we do not know it. The book uses HDBSCAN instead: a density-based method that discovers how many groups the data supports and leaves points that sit in no dense region unassigned (label −1) rather than forcing them into a group. The knob it emphasises is `min_cluster_size`; we also set `min_samples`, as the reference notebook does.

How HDBSCAN works

For every point it measures how far it has to reach to find `min_samples` neighbours, its *core distance*: small in dense regions, large in sparse ones. It then links points by *mutual reachability*, the larger of the two core distances and their actual distance, and builds the hierarchy of groups that appear as the allowed distance grows, pruning anything smaller than `min_cluster_size`. Finally it keeps the groups that persist over the widest range of distances (the *excess of mass* rule), which makes the result stable; points that are never inside a persistent group are outliers. Each member gets a probability that says how firmly it belongs.

k-means, the contrast

k-means picks *k* centres, assigns every point to the nearest one and moves each centre to the mean of its points, repeating until nothing changes. It is fast and gives tidy groups, but it must be told *k*, it draws straight boundaries through whatever shape the data has, and it assigns every point, including the odd ones. Below we run it with the *k* that HDBSCAN discovered, so the two can be compared on the same data.

In [ ]:
```
from hdbscan import HDBSCAN

MIN_CLUSTER_SIZE = 10  # the smallest group HDBSCAN may report
MIN_SAMPLES = 3  # neighbours needed for a point to count as dense; larger values make more outliers
```

```
_clusters_path = SAMPLE_DIR / "clusters.npz"
_clusters_manifest = SAMPLE_DIR / "clusters.json"
_expected_clusters = {
    "reduced_sha256": hashlib.sha256(reduced_embeddings.tobytes()).hexdigest(),
    "min_cluster_size": MIN_CLUSTER_SIZE,
    "min_samples": MIN_SAMPLES,
    "metric": "euclidean",
    "cluster_selection_method": "eom",
    "hdbscan": importlib_metadata.version("hdbscan"),
}
_saved_clusters = (
    json.loads(_clusters_manifest.read_text(encoding="utf-8")) if
_clusters_manifest.exists() else None
)
if _saved_clusters == _expected_clusters and _clusters_path.exists():
    with np.load(_clusters_path) as _saved:
        cluster_labels, cluster_probabilities = _saved["labels"], _saved["probabilities"]
    _origin = f"loaded from `{_clusters_path.name}`"
else:
    _t0 = time.perf_counter()
    _model = HDBSCAN(
        min_cluster_size=MIN_CLUSTER_SIZE, min_samples=MIN_SAMPLES,
        metric="euclidean", cluster_selection_method="eom", core_dist_n_jobs=1,
    ).fit(reduced_embeddings)
    cluster_labels = _model.labels_.astype(np.int64)
    cluster_probabilities = _model.probabilities_.astype(np.float32)
    np.savez(_clusters_path, labels=cluster_labels, probabilities=cluster_probabilities)
    _clusters_manifest.write_text(json.dumps(_expected_clusters, indent=2), encoding="utf-
8")
    _origin = f"fitted now in {time.perf_counter() - _t0:.2f} s and saved to
`{_clusters_path.name}`"
assert cluster_labels.shape == (len(prepared_sample),)
n_clusters = int(cluster_labels.max()) + 1
_outliers = int((cluster_labels == -1).sum())
mo.md(
    f"HDBSCAN (`min_cluster_size={MIN_CLUSTER_SIZE}`, `min_samples={MIN_SAMPLES}`) found
**{n_clusters}** groups; "
    f"**{_outliers}** reviews ({_outliers / len(cluster_labels):.0%}) are outliers with
label −1 · {_origin}. "
    f"Labels stay aligned with `prepared_sample`."
)
```

HDBSCAN (`min_cluster_size=10` , `min_samples=3`) found 37 groups; 155 reviews (16%) are outliers with label −1 · loaded from `clusters.npz` . Labels stay aligned with `prepared_sample` .

In [ ]:

```
# One row per review with its group, plus one row per group with size, strength and
examples.
clustered = prepared_sample[["record_id", "platform", "embedding_text"]].assign(
    cluster=cluster_labels,
    probability=cluster_probabilities,
    x=coordinates_2d[:, 0],
    y=coordinates_2d[:, 1],
)
```

```
def _examples(group):
    return " | ".join(group.sort_values("probability", ascending=False)
["embedding_text"].str.slice(0, 60).head(3))


cluster_summary = (
    clustered[clustered["cluster"] >= 0]
    .groupby("cluster")
    .agg(
        size=("record_id", "size"),
        mean_probability=("probability", "mean"),
        platforms=("platform", lambda s: ", ".join(f"{k} {v}" for k, v in
s.value_counts().items())),
    )
    .assign(share=lambda d: d["size"] / len(clustered))
    .sort_values("size", ascending=False)
    .reset_index()
)
cluster_summary["examples"] = cluster_summary["cluster"].map(lambda c:
_examples(clustered[clustered["cluster"] == c]))
mo.ui.table(
    cluster_summary.round({"mean_probability": 2, "share": 3}),
    label="Groups by size · examples are the three firmest members",
    page_size=10,
)
```

Groups by size · examples are the three firmest members

Search...

cluster int64	size int64	mean_probability float32	platforms string unique: 31	share float64	examples object unique: 37
15	96	0.91000002622604370	google_play 75, app_store 21	0.096	Solo me pase con ustedes por el token físico ...
8	52	0.75999999046325680	google_play 47, app_store 5	0.052	muy fácil de usar | fácil de utilizar | facil de u...
0	44	0.68999999761581420	google_play 44	0.044	excelente | excelente | excelente
4	44	0.50000000000000000	google_play 44	0.044	muy buena | muy buena | muy buena
1	38	0.60000002384185790	google_play 38	0.038	exelente | exelente | gran ap exelente
29	37	0.98000001907348630	google_play 35, app_store 2	0.037	muy eficiente | es rápida y confiable | Es muy ...
31	34	0.81000000238418580	x 32, google_play 2	0.034	Está fallando la aplicación de #BANAMEX? | ...
11	32	0.49000000953674316	google_play 30, app_store 2	0.032	excelente banca, te cuida y protege tus datos ...
20	30	0.77999997138977050	google_play 28, app_store 2	0.030	Muy buena aplicación. | muy buena aplicació...
9	28	0.76999998092651370	google_play 26, app_store 1, x 1	0.028	Bien | me ayuda | bien

37 rows, 6 columns

No selection

In [ ]:

```
def group_map(frame, labels, title, *, highlight=None, height=520):
    """2-D map coloured by group; label -1 is drawn as grey crosses. `highlight` fades
every other group."""
    palette = px.colors.qualitative.Dark24
    labels = np.asarray(labels)
    figure = go.Figure()
    if highlight is None:
        outliers = labels < 0
        figure.add_scatter(
            x=frame["x"][outliers], y=frame["y"][outliers], mode="markers",
name=f"unassigned ({int(outliers.sum())})",
            marker={"size": 5, "color": "#b4b7ac", "symbol": "x"},
            text=frame["embedding_text"][outliers].str.slice(0, 90), hoverinfo="text",
```

```
        )
        order = sorted(set(labels[~outliers].tolist()), key=lambda k: -int((labels ==
k).sum()))
        for i, k in enumerate(order):
            member = labels == k
            figure.add_scatter(
                x=frame["x"][member], y=frame["y"][member], mode="markers", name=f"group
{k} ({int(member.sum())})",
                marker={"size": 6, "color": palette[i % len(palette)]},
                text=frame["embedding_text"][member].str.slice(0, 90), hoverinfo="text",
            )
    else:
        member = labels == highlight
        figure.add_scatter(
            x=frame["x"][~member], y=frame["y"][~member], mode="markers", name="other
reviews",
            marker={"size": 4, "color": "#d3d2c6"}, hoverinfo="skip",
        )
        figure.add_scatter(
            x=frame["x"][member], y=frame["y"][member], mode="markers",
            name="unassigned" if highlight == -1 else f"group {highlight}",
            marker={"size": 7, "color": "#171916", "symbol": "x" if highlight == -1 else
"circle"},
            text=frame["embedding_text"][member].str.slice(0, 90), hoverinfo="text",
        )
    figure.update_layout(
        title={"text": title, "font": {"size": 14}}, height=height,
template="simple_white",
        xaxis_title="UMAP 1", yaxis_title="UMAP 2", legend={"font": {"size": 10}},
    )
    return figure
```

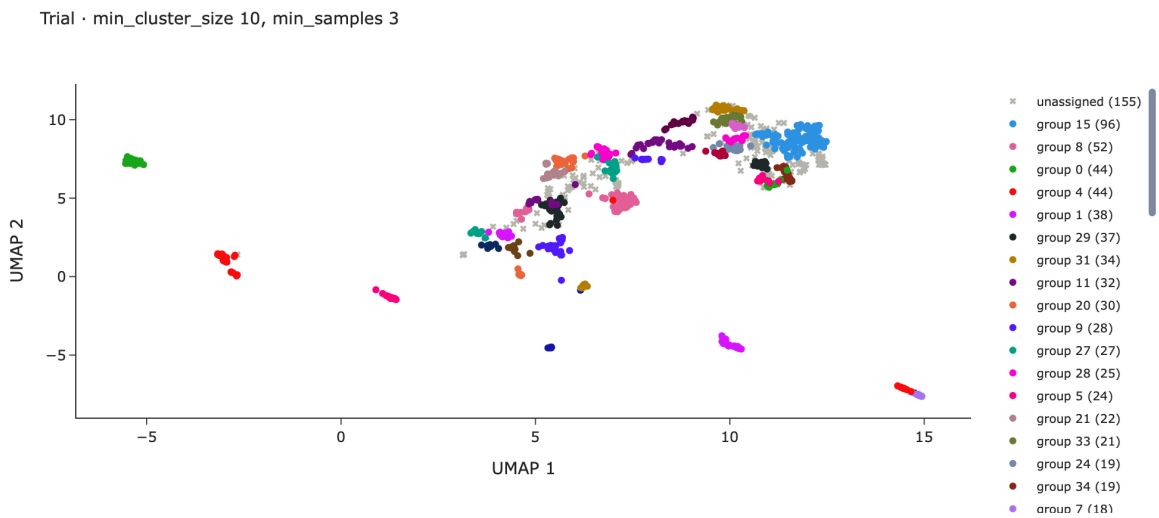
In [ ]:
```
trial_min_cluster_size = mo.ui.slider(start=2, stop=60, step=1, value=MIN_CLUSTER_SIZE,
label="min_cluster_size", show_value=True)
trial_min_samples = mo.ui.slider(start=1, stop=25, step=1, value=MIN_SAMPLES,
label="min_samples", show_value=True)
mo.hstack([trial_min_cluster_size, trial_min_samples], justify="start", gap=2)
```



In [ ]:
```
# Live refit with the slider values. It changes nothing downstream: the saved clustering
above stays the result.
_trial = HDBSCAN(
    min_cluster_size=trial_min_cluster_size.value, min_samples=trial_min_samples.value,
    metric="euclidean", cluster_selection_method="eom", core_dist_n_jobs=1,
).fit_predict(reduced_embeddings)
_sizes = sorted((int((_trial == k).sum()) for k in set(_trial.tolist()) - {-1}),
reverse=True)
_unassigned = int((_trial == -1).sum())
mo.vstack(
    [
        mo.md(
            f"**Try the knobs.** `min_cluster_size` is the smallest group HDBSCAN may
report: raising it merges or "
            f"discards small groups. `min_samples` is how strict the density estimate is:
```

```
raising it turns more points "
            f"into outliers. With **{trial_min_cluster_size.value} /
{trial_min_samples.value}**: "
            f"**{len(_sizes)}** groups, **{_unassigned}** unassigned ({_unassigned /
len(_trial):.0%}), "
            f"largest group {_sizes[0] if _sizes else 0}. The saved result uses
{MIN_CLUSTER_SIZE} / {MIN_SAMPLES}."
        ),
        mo.ui.plotly(group_map(clustered, _trial, f"Trial · min_cluster_size
{trial_min_cluster_size.value}, min_samples {trial_min_samples.value}", height=460)),
    ]
)
```

Try the knobs. `min_cluster_size` is the smallest group HDBSCAN may report: raising it merges or discards small groups. `min_samples` is how strict the density estimate is: raising it turns more points into outliers. With **10 / 3**: **37** groups, **155** unassigned (16%), largest group 96. The saved result uses 10 / 3.



In [ ]:
```
_rows = []
for _mcs in (5, 10, 15, 25):
    for _ms in (1, 3, 5):
        _labels = HDBSCAN(
            min_cluster_size=_mcs, min_samples=_ms, metric="euclidean",
            cluster_selection_method="eom", core_dist_n_jobs=1,
        ).fit_predict(reduced_embeddings)
        _sizes = sorted((int((_labels == k).sum()) for k in set(_labels.tolist()) - {-1}),
reverse=True)
        _rows.append(
            {
                "min_cluster_size": _mcs, "min_samples": _ms, "groups": len(_sizes),
                "unassigned": int((_labels == -1).sum()), "largest_group": _sizes[0] if
_sizes else 0,
            }
        )
cluster_sweep = pd.DataFrame(_rows)
mo.vstack(
    [
        mo.md("**The same knobs, swept systematically.** Twelve settings on the same 5-D
points; the saved result is the 10 / 3 row."),
        mo.ui.table(cluster_sweep, label="HDBSCAN parameter sweep", page_size=12),
```

```
    ]
)
```

The same knobs, swept systematically. Twelve settings on the same 5-D points; the saved result is the 10 / 3 row.

HDBSCAN parameter sweep

Search...

	min_cluster_size int64	min_samples int64	groups int64	unassigned int64	largest_group int64
☐	5	1	74	125	52
☐	5	3	63	158	52
☐	5	5	46	150	77
☐	10	1	38	123	97
☐	10	3	37	155	96
☐	10	5	33	190	69
☐	15	1	22	118	101
☐	15	3	21	125	101
☐	15	5	20	127	99
☐	25	1	16	97	121
☐	25	3	16	103	116
☐	25	5	16	125	99

12 rows, 5 columns *No selection*

In [ ]:

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# k-means told to find as many groups as HDBSCAN discovered, so the two can be compared on
the same points.
kmeans_labels = KMeans(n_clusters=n_clusters, n_init=10,
random_state=42).fit_predict(reduced_embeddings)
_assigned = cluster_labels >= 0
_hdbscan_sizes = sorted((int((cluster_labels == k).sum()) for k in range(n_clusters)),
reverse=True)
_kmeans_sizes = sorted(np.bincount(kmeans_labels).tolist(), reverse=True)
kmeans_comparison = {
    "k": n_clusters,
    "silhouette_kmeans": round(float(silhouette_score(reduced_embeddings, kmeans_labels)),
3),
    "silhouette_hdbscan_assigned":
round(float(silhouette_score(reduced_embeddings[_assigned], cluster_labels[_assigned])),
3),
    "kmeans_sizes": _kmeans_sizes,
}
clustering_comparison = pd.DataFrame(
    [
        {
            "method": "HDBSCAN", "groups": n_clusters, "unassigned":
int((~_assigned).sum()),
            "silhouette": kmeans_comparison["silhouette_hdbscan_assigned"],
"silhouette_over": "assigned points only",
            "smallest_group": _hdbscan_sizes[-1], "largest_group": _hdbscan_sizes[0],
        },
        {
            "method": f"k-means, k = {n_clusters}", "groups": n_clusters, "unassigned": 0,
            "silhouette": kmeans_comparison["silhouette_kmeans"], "silhouette_over": "all
```

```
points",
            "smallest_group": _kmeans_sizes[-1], "largest_group": _kmeans_sizes[0],
        },
    ]
)
mo.vstack(
    [
        mo.md(
            "**k-means as the contrast.** Same points, same number of groups. k-means
assigns every review and "
            "draws straight boundaries through the dense mass of short praise; HDBSCAN
draws around dense regions "
            "and leaves the odd reviews out. The silhouettes are not directly comparable:
HDBSCAN's is computed on "
            "the points it chose to assign."
        ),
        mo.ui.table(clustering_comparison, label="Two clusterings of the same 5-D
points"),
        mo.hstack(
            [
                mo.ui.plotly(group_map(clustered, cluster_labels, "HDBSCAN · grey crosses
are unassigned", height=460)),
                mo.ui.plotly(group_map(clustered, kmeans_labels, f"k-means · k =
{n_clusters}, every review assigned", height=460)),
            ],
            widths="equal",
        ),
    ]
)
```

k-means as the contrast. Same points, same number of groups. k-means assigns every review and draws straight boundaries through the dense mass of short praise; HDBSCAN draws around dense regions and leaves the odd reviews out. The silhouettes are not directly comparable: HDBSCAN's is computed on the points it chose to assign.

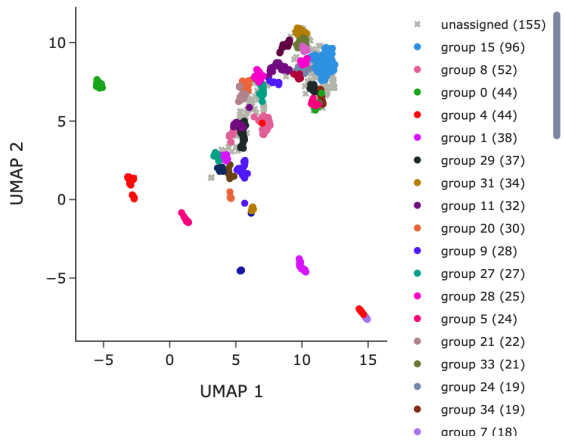
Two clusterings of the same 5-D points

Search...

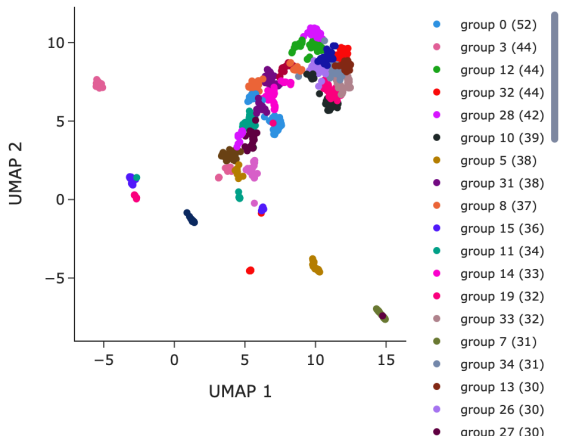
method object	groups int64	unassigned int64	silhouette float64	silhouette_over object	smallest_group int64	largest_group int64
HDBSCAN	37	155	0.664	assigned points only	10	96
k-means, k = 37	37	0	0.591	all points	8	52

2 rows, 7 columns — *No selection*

HDBSCAN · grey crosses are unassigned



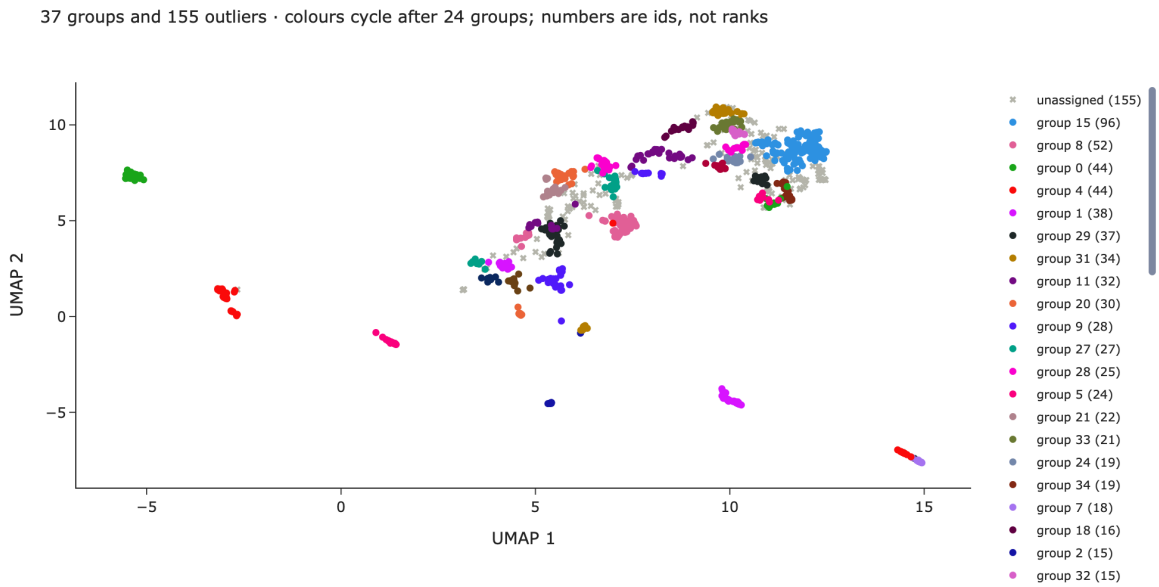
k-means · k = 37, every review assigned



In [ ]:
```
_options = {"All groups": "all", f"Unassigned (−1) · {int((cluster_labels == -1).sum())} reviews": -1}
for _row in cluster_summary.itertuples():
    _options[f"Group {_row.cluster} · {_row.size} reviews"] = int(_row.cluster)
cluster_choice = mo.ui.dropdown(options=_options, value="All groups", label="Show")
cluster_choice
```

Show All groups

In [ ]:
```
_selected = cluster_choice.value
if _selected == "all":
    _figure = group_map(
        clustered, cluster_labels,
        f"{n_clusters} groups and {int((cluster_labels == -1).sum())} outliers · colours cycle after 24 groups; numbers are ids, not ranks",
    )
else:
    _count = int((cluster_labels == _selected).sum())
    _figure = group_map(
        clustered, cluster_labels,
        ("Unassigned reviews" if _selected == -1 else f"Group {_selected}") + f" · {_count} reviews on the 2-D map",
        highlight=_selected,
    )
mo.ui.plotly(_figure)
```



In [ ]:
```
_selected = cluster_choice.value
if _selected == "all":
    _view = mo.md("Pick a group above to read its reviews; the table of groups is in the
cell before the sweep.")
else:
    _members = (
        clustered[clustered["cluster"] == _selected]
        .sort_values("probability", ascending=False)[["record_id", "platform",
"probability", "embedding_text"]]
        .round({"probability": 2})
    )
    _label = "Unassigned reviews, no membership strength" if _selected == -1 else f"Group
{_selected}, firmest members first"
    _view = mo.ui.table(_members, label=_label, page_size=10)
_view
```

Pick a group above to read its reviews; the table of groups is in the cell before the sweep.

In [ ]:
```
# Projection for the Studio "Clusters" page: labels and strengths per row, one record per
group, the sweep
# and the k-means comparison. Texts and 2-D coordinates come from the earlier payloads.
cluster_payload = {
    "params": {
        "min_cluster_size": MIN_CLUSTER_SIZE, "min_samples": MIN_SAMPLES, "metric":
"euclidean",
        "cluster_selection_method": "eom", "hdbscan":
importlib_metadata.version("hdbscan"),
    },
    "counts": {"groups": n_clusters, "unassigned": int((cluster_labels == -1).sum()),
"rows": int(len(cluster_labels))},
    "labels": [int(c) for c in cluster_labels],
    "probabilities": [round(float(p), 2) for p in cluster_probabilities],
    "clusters": [
        {
            "cluster": int(row.cluster), "size": int(row.size), "share":
```

```
round(float(row.share), 4),
            "mean_probability": round(float(row.mean_probability), 3),
            "platforms": {str(k): int(v) for k, v in clustered.loc[clustered["cluster"] ==
row.cluster, "platform"].value_counts().items()},
            "examples": [int(i) for i in clustered[clustered["cluster"] ==
row.cluster].sort_values("probability", ascending=False).index[:3]],
            "centroid_2d": [round(float(v), 3) for v in clustered.loc[clustered["cluster"]
== row.cluster, ["x", "y"]].mean()],
        }
        for row in cluster_summary.itertuples()
    ],
    "sweep": cluster_sweep.to_dict(orient="records"),
    "kmeans": kmeans_comparison,
    "kmeans_labels": [int(c) for c in kmeans_labels],
    "comparison": clustering_comparison.to_dict(orient="records"),
}
mo.md(f"`cluster_payload` ready for the Studio *Clusters* page ·
{len(json.dumps(cluster_payload).encode()) / 1000:,.0f} kB")
```

`cluster_payload` ready for the Studio *Clusters* page · 21 kB

5. Topic modelling

From groups to topics

A group is a set of reviews that say similar things, but reading forty reviews to learn what they share does not scale. Topic modelling describes each group with a handful of keywords, the words that characterise it against the rest of the collection. Keywords are not a label; they are the evidence from which a person (or, later, a language model) names the topic.

What the book says

BERTopic is a modular framework with two halves. The first is exactly what we did: embed, reduce, cluster. The second turns every cluster into a *topic representation*. Each half is a replaceable "Lego block": we feed BERTopic our saved vectors and HDBSCAN groups, so its topics are our groups, renumbered by size (topic 0 is the largest group, topic −1 the unassigned reviews).

The basic block: bag-of-words and c-TF-IDF

Every cluster's reviews are joined into one long document and the words (and two-word phrases) are counted: a bag of words per cluster, not per review. Raw counts would crown words like *muy* or *aplicación* in every topic, so BERTopic weighs them with a class-based TF-IDF. For a term t in cluster c:

$$\text{c-TF-IDF}_{t,c} = \sqrt{\frac{\text{tf}_{t,c}}{\sum_{t'} \text{tf}_{t',c}}} \times \log\left(1 + \frac{A}{f_t}\right)$$

where $\mathrm{tf}_{t,c}$ is how often t appears in cluster c, the square root damps very frequent words (`reduce_frequent_words=True`), f_t is the term's frequency across all clusters, and A is the average number of words per cluster. A term scores high when it is common *inside* the cluster and rare *elsewhere*. The ten highest-scoring terms are the topic's keywords.

Two choices shape the vocabulary: a short Spanish stop-word list (articles, prepositions, pronouns; negations such as *no* are kept on purpose, because *no abre* is a different complaint from *abre*), and bigrams, so *banca móvil* or *no funciona* can be keywords in their own right.

In [ ]:
```
from sklearn.feature_extraction.text import CountVectorizer
from bertopic import BERTopic
from bertopic.backend import BaseEmbedder
from bertopic.cluster import BaseCluster
from bertopic.dimensionality import BaseDimensionalityReduction
from bertopic.vectorizers import ClassTfidfTransformer

SPANISH_STOP_WORDS = sorted({
    "a", "al", "con", "de", "del", "el", "ella", "en", "es", "esta", "este",
    "ha", "la", "las", "lo", "los", "me", "mi", "mis", "para", "por", "que",
    "se", "su", "sus", "un", "una", "unos", "unas", "y", "yo",
})
vectorizer = CountVectorizer(lowercase=True, stop_words=SPANISH_STOP_WORDS, ngram_range=
(1, 2), min_df=1)

# The "Base" blocks are pass-throughs: BERTopic reuses our saved vectors and HDBSCAN
labels instead of
# embedding, reducing and clustering again. Only the keyword half runs here.
topic_model = BERTopic(
    embedding_model=BaseEmbedder(),
    umap_model=BaseDimensionalityReduction(),
    hdbscan_model=BaseCluster(),
    vectorizer_model=vectorizer,
    ctfidf_model=ClassTfidfTransformer(reduce_frequent_words=True),
    top_n_words=10,
    calculate_probabilities=False,
    verbose=False,
)
_topics, _ = topic_model.fit_transform(texts, embeddings=embeddings, y=cluster_labels)
topic_ids = np.asarray(_topics, dtype=int)

# One topic per group and one group per topic; unassigned stays -1.
cluster_to_topic = {int(c): int(t) for c, t in dict(zip(cluster_labels,
topic_ids)).items()}
assert len(cluster_to_topic) == len(set(cluster_to_topic.values()))
assert cluster_to_topic[-1] == -1
topic_info = topic_model.get_topic_info()[["Topic", "Count", "Name", "Representation"]]
mo.vstack(
    [
        mo.md(
            f"**{len(topic_info) - 1}** topics plus the unassigned bucket, from **
{len(topic_model.vectorizer_model.get_feature_names_out()):,}** "
            f"distinct terms (words and bigrams). Topic numbers are BERTopic's, by size;
```

```
group 15 became topic 0."
        ),
        mo.ui.table(topic_info, label="Topics with their top keywords (Name = topic id + first four keywords)", page_size=10),
    ]
)
```

37 topics plus the unassigned bucket, from **8,395** distinct terms (words and bigrams). Topic numbers are BERTopic's, by size; group 15 became topic 0.

Topics with their top keywords (Name = topic id + first four keywords)

Search...

Topic int64	Count int64	Name object unique: 38	Representation object unique:
-1	155	-1_lenta_muy lenta_útil_difícil	["lenta","muy lenta","útil","difícil","ir","led","transfere...
0	96	0_abre_no abre_poder_abrir	["abre","no abre","poder","abrir","aplicación no","solo"...
1	52	1_fácil_muy fácil_fácil usar_usar muy	["fácil","muy fácil","fácil usar","usar muy","usar","rápi...
2	44	2_excelente excelente_excelente_completa excel...	["excelente excelente","excelente","completa excel...
3	44	3_buena muy_muy buena_recomiendo muy_buena	["buena muy","muy buena","recomiendo muy","bue...
4	38	4_exelente_exelente exelente_exelente execelent...	["exelente","exelente exelente","exelente execelent...
5	37	5_eficiente_muy eficiente_rápida eficiente_muy rá...	["eficiente","muy eficiente","rápida eficiente","muy r...
6	34	6_puedo entrar_banamexcontacto_app banamex_...	["puedo entrar","banamexcontacto","app banamex",...
7	32	7_buen banco_banco_siento_siento seguro	["buen banco","banco","siento","siento seguro","banc...
8	30	8_buena aplicación_aplicación aplicación_servici...	["buena aplicación","aplicación aplicación","servici...

38 rows, 4 columns — *No selection*

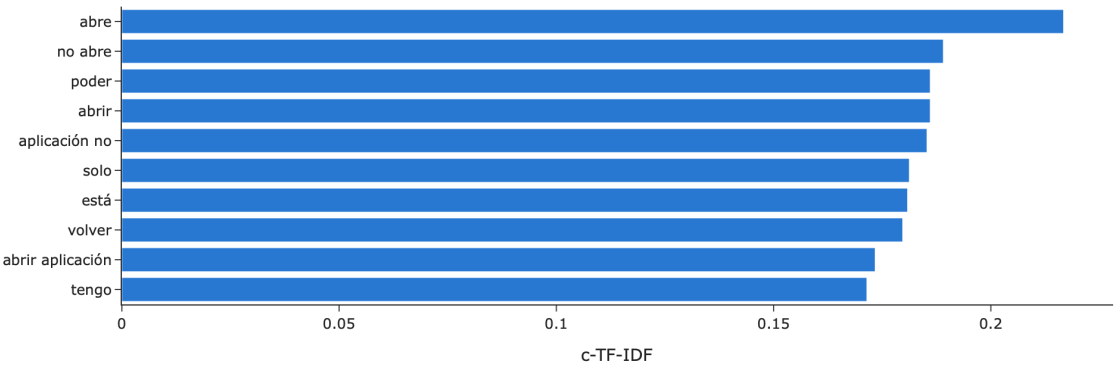
In [ ]:
```
_options = {f"Topic {int(r.Topic)} · {int(r.Count)} reviews · {str(r.Name).split('_', 1)[1][:40]}": int(r.Topic) for r in topic_info.itertuples()}
topic_choice = mo.ui.dropdown(options=_options, value=next(k for k, v in _options.items() if v == 0), label="Inspect")  # start on the largest topic
topic_choice
```

Inspect: Topic 0 · 96 reviews · abre_no abre_poder_abrir

In [ ]:
```
_topic = int(topic_choice.value)
_keywords = pd.DataFrame(topic_model.get_topic(_topic), columns=["term", "weight"])
_bars = go.Figure(go.Bar(x=_keywords["weight"][::-1], y=_keywords["term"][::-1], orientation="h", marker={"color": "#2a78d6"}))
_bars.update_layout(
    title={"text": f"Topic {_topic} · c-TF-IDF weight of its ten keywords", "font": {"size": 14}},
    height=360, template="simple_white", margin={"l": 140, "r": 20, "t": 50, "b": 40},
xaxis_title="c-TF-IDF",
)
_members = clustered.assign(topic=topic_ids)
_members = _members[_members["topic"] == _topic].sort_values("probability", ascending=False)
_representative = topic_model.get_representative_docs(_topic) or []
mo.vstack(
    [
        mo.ui.plotly(_bars),
        mo.md(
            f"**{len(_members)} reviews** in topic {_topic}"
```

```
            + ("" if _topic == -1 else f" (our group {next(c for c, t in
cluster_to_topic.items() if t == _topic)})")
            + ". BERTopic's **representative reviews** are the ones whose own bag of words
is closest to the topic's c-TF-IDF vector:"
        ),
        mo.md("\n".join(f"- {d}" for d in _representative[:3]) if _representative else
"_none_"),
        mo.ui.table(_members[["record_id", "platform", "probability",
"embedding_text"]].round({"probability": 2}), label="All reviews in this topic, firmest
members first", page_size=8),
    ]
)
```

Topic 0 · c-TF-IDF weight of its ten keywords



96 reviews in topic 0 (our group 15). BERTopic's **representative reviews** are the ones whose own bag of words is closest to the topic's c-TF-IDF vector:

- Ciertamente, el desempeño de su app es horrendo. Siempre saltan errores, como que "se pierde el contacto con el servidor" y no te deja abrir tu tarjeta digital de débito, inicios de sesión no exitosos, lags y un sin fin de osas que no deberían suceder. En este sentido, las apps de SOFIPOS como Nu y Klar los patean pero con muchos moretones. Además, son errores comunes, porque suceden desde hace años y persisten a la fecha.
- muy mala aplicación últimamente no me deja acceder desde ase como 20 días a estado fallando mucho pues entro ala aplicación y solo se pone obscura la pantalla antes era muy buena solucionen ese problema pues para poder entrar tengo que estar reiniciando el teléfono acá rato y si bien logro acceder ala aplicación si no seguir intentando una y otra ves y es muy molesto
- Una completa basura, media hora para abrir una cuenta de debito por que el sistema no quería aceptar bien mis datos y media sacaba a cada rato, ya que por fin la cree se traba y tarda en abrir, ni siquiera me carga la opción para enviar la tarjeta a mi domicilio tengo que ir por ella a sucursal, para que le ponen entrega a domicilio entonces?, la app es confusa y nada intuitiva, pésima aplicación pésimo servicio , parece que te hacen un favor al abrir tu cuenta, ni siquiera tiene un chat de atención al cliente y el numero de teléfono nunca contestan, la única ves que contestaron me pidieron ir a sucursal solo para enviarme mi tarjeta!! Este banco es ideal si te gusta ir a perder tu tiempo dando vueltas diario a sucursales.

All reviews in this topic, firmest members first

Search...

	Index0 int64	record_id string unique: 96	platform string unique: 2	probability float32	embedding_text string unique: 96
☐	**1**	play:com.citibanamex.banamexmobile:85203f...	google_play	1	Solo me pase con ustedes por el token físico p...
☐	**548**	appbot:498483038:3729352291	app_store	1	No está funcionando esta aplicación. Es un de...
☐	**693**	appbot:498483038:3709625634	app_store	1	Hoy sábado tiene horas que no funciona y des...
☐	**668**	appbot:498483038:3746325152	app_store	1	Llevo una semana intentando registrarme en l...
☐	**656**	play:com.citibanamex.banamexmobile:18e4cd...	google_play	1	La aplicación lleva dos semanas sin poder siq...
☐	**653**	play:com.citibanamex.banamexmobile:14b731...	google_play	1	llevo todo el día intentando entrar a la app y n...
☐	**646**	play:com.citibanamex.banamexmobile:488ac3...	google_play	1	La peor de las apps no abre solo se queda car...
☐	**634**	play:com.citibanamex.banamexmobile:71c77a...	google_play	1	muy mala aplicación últimamente no me deja ...

96 rows, 4 columns *No selection*

In [ ]:
```
# Reproduce the formula by hand for the inspected topic, term by term, and compare with
BERTopic's weights.
_topic = int(topic_choice.value)
_per_topic = pd.Series(texts).groupby(topic_ids).apply(" ".join)
_bow = topic_model.vectorizer_model.transform(_per_topic.tolist())
_terms = topic_model.vectorizer_model.get_feature_names_out()
_term_index = {t: i for i, t in enumerate(_terms)}
_words_per_topic = np.asarray(_bow.sum(axis=1)).ravel()
_f_t = np.asarray(_bow.sum(axis=0)).ravel()
_A = int(_words_per_topic.mean())
_row = list(_per_topic.index).index(_topic)
```

```
_rows = []
for _term, _weight in topic_model.get_topic(_topic):
    _i = _term_index[_term]
    _tf = int(_bow[_row, _i])
    _idf = float(np.log(1 + _A / _f_t[_i]))
    _rows.append(
        {
            "term": _term, "tf in topic": _tf, "words in topic":
int(_words_per_topic[_row]),
            "f_t everywhere": int(_f_t[_i]), "idf = log(1 + A/f_t)": round(_idf, 3),
            "by hand": round(float(np.sqrt(_tf / _words_per_topic[_row]) * _idf), 4),
"BERTopic": round(float(_weight), 4),
        }
    )
ctfidf_check = pd.DataFrame(_rows)
assert np.allclose(ctfidf_check["by hand"], ctfidf_check["BERTopic"], atol=1e-3)
mo.vstack(
    [
        mo.md(f"**The formula, by hand**, for topic {_topic}: A = {_A} words per topic on
average. The last two columns agree."),
        mo.ui.table(ctfidf_check, label="c-TF-IDF recomputed from the counts"),
    ]
)
```

The formula, by hand, for topic 0: A = 435 words per topic on average. The last two columns agree.

c-TF-IDF recomputed from the counts

Search...

term object	tf in topic int64	words in topic int64	f_t everywhere int64	idf = log(1 + A/f_t) float64	by hand float64	BERTopic float64
abre	19	3,431	25	2.912	0.2167	0.2167
no abre	11	3,431	16	3.339	0.1891	0.1891
poder	14	3,431	25	2.912	0.1860	0.1860
abrir	14	3,431	25	2.912	0.1860	0.1860
aplicación no	9	3,431	12	3.618	0.1853	0.1853
solo	15	3,431	30	2.741	0.1812	0.1812
está	19	3,431	42	2.430	0.1808	0.1808
volver	6	3,431	6	4.297	0.1797	0.1797
abrir aplicación	6	3,431	7	4.145	0.1734	0.1734
tengo	18	3,431	45	2.367	0.1715	0.1715

10 rows, 7 columns

No selection

In [ ]:

```
# Projection for the Studio "Topics" page: every topic with its keywords and weights, the
group it came from,
# representative reviews (as row indices), the vocabulary choices, and the numbers behind
the formula
# (counts inside the topic, counts everywhere, idf, weight) for the top terms by count and
by weight.
_rep_index = {t: i for i, t in enumerate(texts)}
_per_topic = pd.Series(texts).groupby(topic_ids).apply(" ".join)
_bow = topic_model.vectorizer_model.transform(_per_topic.tolist()).tocsr()
_terms = topic_model.vectorizer_model.get_feature_names_out()
_words_per_topic = np.asarray(_bow.sum(axis=1)).ravel()
_f_t = np.asarray(_bow.sum(axis=0)).ravel()
_A = int(_words_per_topic.mean())
_idf = np.log(1 + _A / _f_t)
```

```
_weights = topic_model.c_tf_idf_.tocsr()  # rows follow the sorted topic ids, like
_per_topic


def _walk(row, topic):
    counts = _bow.getrow(row).toarray().ravel()
    weights = _weights.getrow(row).toarray().ravel()
    by_count = np.argsort(-counts, kind="stable")
    by_weight = np.argsort(-weights, kind="stable")
    rank_count = np.empty(len(counts), dtype=int); rank_count[by_count] = np.arange(1,
len(counts) + 1)
    rank_weight = np.empty(len(weights), dtype=int); rank_weight[by_weight] = np.arange(1,
len(weights) + 1)
    chosen = sorted(set(by_count[:12].tolist()) | set(by_weight[:10].tolist()), key=lambda
i: -weights[i])
    return {
        "topic": int(topic),
        "words": int(_words_per_topic[row]),
        "terms": [
            {
                "term": str(_terms[i]), "tf": int(counts[i]), "f_t": int(_f_t[i]), "idf":
round(float(_idf[i]), 4),
                "weight": round(float(weights[i]), 4), "rank_count": int(rank_count[i]),
"rank_weight": int(rank_weight[i]),
            }
            for i in chosen if counts[i] > 0
        ],
    }


topic_payload = {
    "params": {
        "representation": "c-TF-IDF", "reduce_frequent_words": True, "ngram_range": [1,
2], "top_n_words": 10,
        "stop_words": SPANISH_STOP_WORDS, "vocabulary": int(len(_terms)),
        "bertopic": importlib_metadata.version("bertopic"),
    },
    "counts": {"topics": int(len(topic_info) - 1), "unassigned": int((topic_ids ==
-1).sum()), "rows": int(len(topic_ids))},
    "A": _A,
    "topic_of_row": [int(t) for t in topic_ids],
    "topics": [
        {
            "topic": int(r.Topic), "count": int(r.Count), "name": str(r.Name),
            "cluster": next(c for c, t in cluster_to_topic.items() if t == int(r.Topic)),
            "keywords": [{"term": w, "weight": round(float(s), 4)} for w, s in
topic_model.get_topic(int(r.Topic)) if w],
            "representative": [_rep_index[d] for d in
(topic_model.get_representative_docs(int(r.Topic)) or [])[:3] if d in _rep_index],
        }
        for r in topic_info.itertuples()
    ],
    "walk": [_walk(row, topic) for row, topic in enumerate(_per_topic.index)],
}
mo.md(f"`topic_payload` ready for the Studio *Topics* page ·
{len(json.dumps(topic_payload, ensure_ascii=False).encode()) / 1000:,.0f} kB")
```

`topic_payload` ready for the Studio *Topics* page · 99 kB

6. Naming the topics

Three Lego blocks on top of c-TF-IDF

The bag-of-words keywords are fast and transparent, but they know nothing about meaning. BERTopic lets us stack representation blocks that take the c-TF-IDF candidates for each topic and improve them, once per topic rather than once per review. The three in the book are independent; we run all three.

- KeyBERT-inspired: embeds the candidate words and the topic's representative reviews with our encoder and reranks the words by similarity to the topic. Same vocabulary, better order: *app falla* over *abre*.
- Maximal marginal relevance (MMR): picks keywords one at a time, each time trading similarity to the topic against similarity to the words already chosen (`diversity` sets the trade-off), so *excelente* and *excelente excelente* stop crowding the list. We chain it after KeyBERT with 30 candidates.
- Text generation: the only block that produces something new. For each topic it sends the keywords and a few representative reviews to a language model and asks for a label; the model never sees the whole corpus. We reuse the demo's Spanish prompt (`voc_demo.llm.topic_interpretation`): reviews are declared untrusted data, the answer must be JSON with a proposed feature, a description, the ids of the reviews that support it, and an assessment (`single_feature` , `mixed` , `unclear`).

Cached, like everything else

Reranked keywords are saved beside the embeddings. Labels are saved per topic, keyed on the model, the prompt version and a fingerprint of the exact evidence sent, so a rerun never calls the API unless the evidence changed; only the Name the topics button triggers calls, and only for topics not yet named.

In [ ]:
```
from copy import deepcopy
from bertopic.backend._sentencetransformers import SentenceTransformerBackend
from bertopic.representation import KeyBERTInspired, MaximalMarginalRelevance

MMR_DIVERSITIES = (0.3, 0.7)
_inputs_fingerprint = hashlib.sha256("".join(prepared_sample["input_hash"]).encode("utf-
8")).hexdigest()
_repr_path = SAMPLE_DIR / "representations.json"
_expected_repr = {
    "inputs_sha256": _inputs_fingerprint,
    "topics_sha256": hashlib.sha256(json.dumps([int(t) for t in
topic_ids]).encode()).hexdigest(),
    "encoder": MODEL_ID, "revision": MODEL_REVISION, "candidates": 30, "top_n_words": 10,
    "mmr_diversities": list(MMR_DIVERSITIES), "bertopic":
```

```
importlib_metadata.version("bertopic"),
}
_saved_repr = json.loads(_repr_path.read_text(encoding="utf-8")) if _repr_path.exists()
else None
if _saved_repr and _saved_repr.get("manifest") == _expected_repr:
    topic_aspects = {a: {int(t): words for t, words in d.items()} for a, d in
_saved_repr["aspects"].items()}
    _origin = f"loaded from `{_repr_path.name}`"
else:
    _t0 = time.perf_counter()
    _model = deepcopy(topic_model)  # update_topics mutates the model it receives
    _model.embedding_model = SentenceTransformerBackend(encoder)
    _model.update_topics(
        texts, top_n_words=10, vectorizer_model=_model.vectorizer_model,
ctfidf_model=_model.ctfidf_model,
        representation_model={
            "keybert": KeyBERTInspired(top_n_words=10, random_state=42),
            **{
                f"mmr_{d}": [KeyBERTInspired(top_n_words=30, random_state=42),
MaximalMarginalRelevance(diversity=d)]
                for d in MMR_DIVERSITIES
            },
        },
    )
    assert _model.topics_ == topic_model.topics_
    topic_aspects = {
        a: {int(t): [[w, round(float(s), 4)] for w, s in words if w] for t, words in
d.items()}
        for a, d in _model.topic_aspects_.items()
    }
    _repr_path.write_text(json.dumps({"manifest": _expected_repr, "aspects":
topic_aspects}, ensure_ascii=False, indent=1), encoding="utf-8")
    _origin = f"computed in {time.perf_counter() - _t0:.0f} s and saved to
`{_repr_path.name}`"
topic_aspects["ctfidf"] = {int(t): [[w, round(float(s), 4)] for w, s in
topic_model.get_topic(int(t)) if w] for t in topic_info["Topic"]}


def _top(aspect, t, n=5):
    return " · ".join(w for w, _ in topic_aspects[aspect].get(int(t), [])[:n])


representation_table = pd.DataFrame(
    [
        {"topic": int(r.Topic), "count": int(r.Count), "c-TF-IDF": _top("ctfidf",
r.Topic), "KeyBERT": _top("keybert", r.Topic),
         **{f"KeyBERT → MMR {d}": _top(f"mmr_{d}", r.Topic) for d in MMR_DIVERSITIES}}
        for r in topic_info.itertuples() if int(r.Topic) >= 0
    ]
)
mo.vstack(
    [
        mo.md(f"Reranked keywords for {len(representation_table)} topics, {_origin}. The
label block uses the **KeyBERT → MMR {MMR_DIVERSITIES[0]}** list."),
        mo.ui.table(representation_table, label="First five keywords per representation",
page_size=10),
```

```
    ]
)
```

Reranked keywords for 37 topics, loaded from `representations.json`. The label block uses the **KeyBERT → MMR 0.3** list.

First five keywords per representation

Search...

	topic int64	count int64	c-TF-IDF object unique: 37	KeyBERT object unique: 37	KeyBERT → MMR 0.3 object unique: 37
☐	0	96	abre · no abre · poder · abrir · aplicación no	app falla · problemas app · entrar app · app · a...	app falla · problemas app
☐	1	52	fácil · muy fácil · fácil usar · usar muy · usar	fácil usar · fácil utilizar · facil usar · fácil uso · ...	fácil usar · fácil utilizar · f
☐	2	44	excelente excelente · excelente · completa exc...	excelente · excelentev · excelente excelente · ...	excelente · excelentev · e
☐	3	44	buena muy · muy buena · recomiendo muy · b...	muy buena · muy bueno · buena buenísima · b...	muy bueno · buena bueni
☐	4	38	exelente · exelente exelente · exelente execel...	exelente exelente · exelente eselete · exelente...	exelente exelente · exele
☐	5	37	eficiente · muy eficiente · rápida eficiente · mu...	muy eficiente · rápido eficiente · eficiente muy...	muy eficiente · rápido efic
☐	6	34	puedo entrar · banamexcontacto · app baname...	app banamex · aplicación banamex · entrar ap...	app banamex · aplicación
☐	7	32	buen banco · banco · siento · siento seguro · b...	excelente banca · buen banco · excelente banc...	excelente banca · buen ba
☐	8	30	buena aplicación · aplicación aplicación · servi...	aplicación bueno · buena aplicación · excelent...	aplicación bueno · buena
☐	9	28	bien bien · patita · destrozada · aceptable · má...	susanita muy · lata toma · cosa muy · exelente...	susanita muy · lata toma

37 rows, 6 columns — *No selection*

In [ ]:

```
from voc_demo.llm.topic_interpretation import PROMPT_VERSION, RESPONSE_SCHEMA,
SYSTEM_PROMPT, fingerprint
from voc_demo.llm.openrouter import create_openrouter_client
from jsonschema import Draft202012Validator, ValidationError

LABEL_ASPECT = f"mmr_{MMR_DIVERSITIES[0]}"
EVIDENCE_REVIEWS = 6
EVIDENCE_CHARS = 900
_rows = clustered.assign(topic=topic_ids)


def _evidence_rows(t):
    """Representative reviews first, then the firmest members, without duplicates."""
    members = _rows[_rows["topic"] == t].sort_values("probability", ascending=False)
    representative = [i for i in topic_payload["topics"][[x["topic"] for x in
topic_payload["topics"]].index(t)]["representative"]]
    order = list(dict.fromkeys(representative + members.index.tolist()))
[:EVIDENCE_REVIEWS]
    # Short aliases instead of record ids: the model must copy them back, and it mangles
36-character UUIDs.
    return [
        {"id": f"e{n}", "text": texts[i][:EVIDENCE_CHARS], "truncated": len(texts[i]) >
EVIDENCE_CHARS}
        for n, i in enumerate(order, start=1)
    ], {f"e{n}": str(prepared_sample.loc[i, "record_id"]) for n, i in enumerate(order,
start=1)}


_built = {int(t): _evidence_rows(int(t)) for t in topic_info["Topic"] if int(t) >= 0}
evidence_record_ids = {t: ids for t, (_, ids) in _built.items()}  # alias -> record id,
never sent to the model
label_requests = [
    {
```

```
        "topic_run_id": "prototypeV0-workflow01",
        "topic_id": int(t),
        "dataset_release_id": snapshot.info["release_id"],
        "keywords": [w for w, _ in topic_aspects[LABEL_ASPECT][int(t)]],
        "evidence": _built[int(t)][0],
    }
    for t in topic_info["Topic"] if int(t) >= 0
]
mo.vstack(
    [
        mo.md(
            f"**{len(label_requests)}** requests prepared, one per topic: the
{LABEL_ASPECT} keywords plus up to "
            f"{EVIDENCE_REVIEWS} reviews of at most {EVIDENCE_CHARS} characters. Example,
topic 0:"
        ),
        mo.ui.code_editor(json.dumps(label_requests[0], ensure_ascii=False, indent=2),
language="json", disabled=True, max_height=320),
    ]
)
```

37 requests prepared, one per topic: the mmr_0.3 keywords plus up to 6 reviews of at most 900 characters. Example, topic 0:


```
{
  "topic_run_id": "prototypeV0-workflow01",
  "topic_id": 0,
  "dataset_release_id": "b7a61a8b-27d8-46a7-b2ce-67d97268b741",
  "keywords": [
    "app falla",
    "problemas app",
    "entrar app",
    "app",
    "apps",
    "app no",
    "abrir aplicación",
    "aplicación no",
    "problemas abrir",
    "reiniciar"
  ],
```


In [ ]:
```
name_topics = mo.ui.run_button(label="Name the topics with google/gemini-2.5-flash · one
call per unnamed topic, answers cached")
name_topics
```

Name the topics with google/gemini-2.5-flash · one call per unnamed topic, answers cached

In [ ]:
```
LABEL_MODEL = "google/gemini-2.5-flash"
_labels_path = SAMPLE_DIR / "labels.json"
_store = json.loads(_labels_path.read_text(encoding="utf-8")) if _labels_path.exists()
else None
if _store and any("record_id" in e for a in _store.get("attempts", {}).values() for e in
a["request"]["evidence"]):
    # Answers from the earlier evidence format (full record ids) are kept aside, not
reused.
    _labels_path.rename(_labels_path.with_name("labels.uuid-ids.json"))
    _store = None
if not _store or _store.get("model") != LABEL_MODEL or _store.get("prompt_version") !=
PROMPT_VERSION:
    _store = {"model": LABEL_MODEL, "prompt_version": PROMPT_VERSION, "attempts": {},
"failures": []}
```

```
def request_label(payload):
    """One explicit request, mirroring
voc_demo.llm.topic_interpretation.request_interpretation."""
    attempt = {
        "created_at": pd.Timestamp.now(tz="UTC").isoformat(timespec="seconds"),
        "prompt_version": PROMPT_VERSION, "requested_model": LABEL_MODEL,
        "request": payload, "evidence_sha256": fingerprint(payload),
        "generation_parameters": {"temperature": 0, "max_tokens": 800},
        "status": "failed", "response": None, "raw_response": None, "error": None,
    }
    try:
        with create_openrouter_client(max_retries=0) as client:
            reply = client.chat.completions.create(
                model=LABEL_MODEL,
                messages=[{"role": "system", "content": SYSTEM_PROMPT}, {"role": "user",
"content": json.dumps(payload, ensure_ascii=False)}],
                response_format={"type": "json_schema", "json_schema": {"name":
"app_feature", "strict": True, "schema": RESPONSE_SCHEMA}},
                extra_body={"provider": {"require_parameters": True}},
                **attempt["generation_parameters"],
            )
        attempt["returned_model"] = reply.model
        attempt["usage"] = reply.usage.model_dump(mode="json") if reply.usage else None
        choice = reply.choices[0]
        attempt["raw_response"] = choice.message.content
        if choice.finish_reason != "stop" or getattr(choice.message, "refusal", None):
            raise ValueError("incomplete or refused response")
        response = json.loads(choice.message.content)
        Draft202012Validator(RESPONSE_SCHEMA).validate(response)
        if not set(response["supporting_review_ids"]) <= {e["id"] for e in
payload["evidence"]}:
            raise ValueError("the model cited a review it was not given")
        attempt["response"] = response
        attempt["evidence_record_ids"] = evidence_record_ids[payload["topic_id"]]
        attempt["supporting_record_ids"] = [evidence_record_ids[payload["topic_id"]][a]
for a in response["supporting_review_ids"]]
        attempt["status"] = "completed"
    except (ValueError, TypeError, IndexError, ValidationError, json.JSONDecodeError) as
exc:
        attempt["error"] = {"category": type(exc).__name__, "reason": str(exc)[:200]}
    except Exception as exc:  # noqa: BLE001 - keep the failure, never the credentials
        attempt["error"] = {"category": type(exc).__name__, "reason": "request_failed"}
    return attempt


_pending = [p for p in label_requests if fingerprint(p) not in _store["attempts"]]
_called = 0
if _pending and name_topics.value:
    for _payload in _pending:
        _attempt = request_label(_payload)
        _called += 1
        if _attempt["status"] == "completed":
            _store["attempts"][_attempt["evidence_sha256"]] = _attempt
        else:
```

```
            _store["failures"].append(_attempt)
        _labels_path.write_text(json.dumps(_store, ensure_ascii=False, indent=1),
encoding="utf-8")  # after every call
    _pending = [p for p in label_requests if fingerprint(p) not in _store["attempts"]]

label_attempts = {p["topic_id"]: _store["attempts"][fingerprint(p)] for p in
label_requests if fingerprint(p) in _store["attempts"]}
label_failures = list(_store["failures"])
topic_labels = pd.DataFrame(
    [
        {
            "topic": t, "count": int(topic_info.loc[topic_info["Topic"] == t,
"Count"].iloc[0]),
            "label": a["response"]["proposed_feature"] or "—", "assessment": a["response"]
["assessment"],
            "description": a["response"]["topic_description"], "cited": len(a["response"]
["supporting_review_ids"]),
            "keywords": " · ".join(a["request"]["keywords"][:5]),
        }
        for t, a in sorted(label_attempts.items())
    ]
)
_status = (
    f"**{len(label_attempts)}** of {len(label_requests)} topics named"
    + (f" · {_called} calls made now" if _called else " · no calls made in this run")
    + (f" · **{len(_pending)} still unnamed: press the button above**" if _pending else " ·
all cached in `labels.json`")
    + (f" · {len(_store['failures'])} failed attempts kept for inspection" if
_store["failures"] else "")
)
mo.vstack(
    [
        mo.md(_status),
        mo.ui.table(topic_labels, label="Labels proposed by the model, with its own
assessment", page_size=10) if len(topic_labels) else mo.md("_No labels yet._"),
    ]
)
```

37 of 37 topics named · no calls made in this run · all cached in **`labels.json`**

Labels proposed by the model, with its own assessment

Search...

	topic int64	count int64	label object unique: 11	assessment object unique: 3	description object unique: 37	cit int
☐	0	96	Mejorar la estabilidad y el rendimiento de la a...	single_feature	Los usuarios reportan problemas recurrentes c...	
☐	1	52	Facilidad de uso de la aplicación	single_feature	Los usuarios valoran que la aplicación sea fáci...	
☐	2	44	–	unclear	Las reseñas expresan una opinión positiva gen...	
☐	3	44	–	unclear	Las reseñas expresan una opinión general pos...	
☐	4	38	–	mixed	Las reseñas expresan satisfacción general con ...	
☐	5	37	Mejorar la velocidad de las transacciones	single_feature	Los usuarios valoran la eficiencia y rapidez de ...	
☐	6	34	Mejorar la estabilidad y el acceso a la aplicación	single_feature	Los usuarios experimentan problemas para in...	
☐	7	32	Funcionalidad bancaria general	single_feature	Los usuarios expresan satisfacción con la aplic...	
☐	8	30	–	unclear	Las reseñas expresan satisfacción general con ...	
☐	9	28	–	mixed	Las reseñas son muy variadas y no se centran ...	

37 rows, 7 columns *No selection*

In [ ]:
```
_named = {f"Topic {t} · {a['response']['proposed_feature'] or a['response']
['assessment']}": t for t, a in sorted(label_attempts.items())}
mo.stop(not _named, mo.md("No labels to inspect yet."))
label_choice = mo.ui.dropdown(options=_named, value=next(iter(_named)), label="Inspect the
exchange for")
label_choice
```

Inspect the exchange for Topic 0 · Mejorar la estabilidad y el rendimiento de la aplicación

In [ ]:
```
_attempt = label_attempts[int(label_choice.value)]
_cited = set(_attempt["response"]["supporting_review_ids"])
_evidence = _attempt["request"]["evidence"]
mo.vstack(
    [
        mo.md(
            f"### Topic {_attempt['request']['topic_id']} · **{_attempt['response']
['proposed_feature'] or '(no single feature)'}**\n\n"
            f"{_attempt['response']['topic_description']}\n\n"
            f"Assessment: `{_attempt['response']['assessment']}` · model
`{_attempt.get('returned_model', LABEL_MODEL)}` · "
            f"tokens {(_attempt.get('usage') or {}).get('total_tokens', '?')} ·
{_attempt['created_at']}"
        ),
        mo.accordion(
            {
                "System prompt (shared by every call)":
mo.md(f"```\n{SYSTEM_PROMPT}\n```"),
                "User message (this topic's evidence)":
mo.ui.code_editor(json.dumps(_attempt["request"], ensure_ascii=False, indent=2),
language="json", disabled=True, max_height=360),
                "Raw answer": mo.ui.code_editor(_attempt["raw_response"] or "",
language="json", disabled=True, max_height=240),
            }
        ),
        mo.md("**Evidence sent, cited reviews in bold:**"),
```

```
        mo.md("\n".join(f"- `{e['id']}` {'**' if e['id'] in _cited else ''}{e['text']}
{'**' if e['id'] in _cited else ''}" for e in _evidence)),
    ]
)
```

Topic 0 · **Mejorar la estabilidad y el rendimiento de la aplicación**

Los usuarios reportan problemas recurrentes con la aplicación, incluyendo errores de conexión al servidor, fallos al iniciar sesión, lentitud, bloqueos y la imposibilidad de abrir la aplicación, lo que afecta la experiencia general del usuario y el acceso a funcionalidades esenciales.

Assessment: `single_feature` · model `google/gemini-2.5-flash` · tokens 985 · 2026-10-02T20:50:03+00:00

System prompt (shared by every call)

User message (this topic's evidence)

Raw answer

Evidence sent, cited reviews in bold:

- e1 **Ciertamente, el desempeño de su app es horrendo. Siempre saltan errores, como que "se pierde el contacto con el servidor" y no te deja abrir tu tarjeta digital de débito, inicios de sesión no exitosos, lags y un sin fin de osas que no deberían suceder. En este sentido, las apps de SOFIPOS como Nu y Klar los patean pero con muchos moretones. Además, son errores comunes, porque suceden desde hace años y persisten a la fecha.**
- e2 **muy mala aplicación últimamente no me deja acceder desde ase como 20 días a estado fallando mucho pues entro ala aplicación y solo se pone obscura la pantalla antes era muy buena solucionen ese problema pues para poder entrar tengo que estar reiniciando el teléfono acá rato y si bien logro acceder ala aplicación si no seguir intentando una y otra ves y es muy molesto**
- e3 **Una completa basura, media hora para abrir una cuenta de debito por que el sistema no quería aceptar bien mis datos y media sacaba a cada rato, ya que por fin la cree se traba y tarda en abrir, ni siquiera me carga la opción para enviar la tarjeta a mi domicilio tengo que ir por ella a sucursal, para que le ponen entrega a domicilio entonces?, la app es confusa y nada intuitiva, pésima aplicación pésimo servicio , parece que te hacen un favor al abrir tu cuenta, ni siquiera tiene un chat de atención al cliente y el numero de teléfono nunca contestan, la única ves que contestaron me pidieron ir a sucursal solo para enviarme mi tarjeta!! Este banco es ideal si te gusta ir a perder tu tiempo dando vueltas diario a sucursales.**
- e4 **Solo me pase con ustedes por el token físico pero en verdad que horrible está su aplicación, no puede ser que no tengan dinero para pagarle a buenos desarrolladores y hagan bien sus cosas, que nefasta aplicación, no abre, se traba, es lenta, tiene un horrible diseño, mal manejo de colores, en verdad no les da vergüenza?**
- e5 **No está funcionando esta aplicación. Es un desastre**
- e6 **Hoy sábado tiene horas que no funciona y después de solo aparecer un mensaje de estamos trabajando me pide actualizar lego Me envió a App Store y ahí solo aparece la opción de abrir. No hay actualizaciones por instalar**

In [ ]:
```
# Projection for the Studio "Naming" page: every representation per topic, the label
exchanges (prompt, evidence,
# answer) and the cited reviews as row indices. No credentials; the system prompt is
public project text.
_row_of_record = {str(r): i for i, r in enumerate(prepared_sample["record_id"])}
naming_payload = {
    "model": LABEL_MODEL,
    "prompt_version": PROMPT_VERSION,
    "system_prompt": SYSTEM_PROMPT,
    "label_aspect": LABEL_ASPECT,
    "evidence_rules": {"reviews": EVIDENCE_REVIEWS, "chars": EVIDENCE_CHARS},
    "aspects": {a: {str(t): words for t, words in d.items()} for a, d in
topic_aspects.items()},
    "counts": {"topics": len(label_requests), "named": len(label_attempts), "failed":
len(label_failures)},
    "labels": [
```

```
        {
            "topic": t,
            "label": a["response"]["proposed_feature"],
            "assessment": a["response"]["assessment"],
            "description": a["response"]["topic_description"],
            "keywords": a["request"]["keywords"],
            "evidence": [_row_of_record[a["evidence_record_ids"][e["id"]]] for e in
a["request"]["evidence"]],
            "supporting": [_row_of_record[r] for r in a["supporting_record_ids"] if r in
_row_of_record],
            "user_message": json.dumps(a["request"], ensure_ascii=False, indent=2),
            "raw_response": a["raw_response"],
            "returned_model": a.get("returned_model"),
            "tokens": (a.get("usage") or {}).get("total_tokens"),
            "created_at": a["created_at"],
        }
        for t, a in sorted(label_attempts.items())
    ],
}
mo.md(f"`naming_payload` ready for the Studio *Naming* page ·
{len(json.dumps(naming_payload, ensure_ascii=False).encode()) / 1000:,.0f} kB ·
{len(naming_payload['labels'])} labels")
```

`naming_payload` ready for the Studio *Naming* page · 150 kB · 37 labels

7. Sentiment analysis

What we measure

Topics say what people talk about; sentiment says how they feel while saying it. We classify each review as a whole into negative, neutral or positive with RoBERTuito, a Spanish RoBERTa fine-tuned for sentiment by the pysentimiento project, or an existing domain-adapted checkpoint selected below. Training and held-out evaluation are handled separately in `workflow04_transfer_learning.py. This notebook performs inference on CPU. Reviews pass through the model authors' Spanish preprocessing and are truncated at 128 tokens; the model's output is three probabilities and the label is their argmax. Only the review text enters the classifier: never the topic's keywords or the generated label.

How it joins the topics

Sentiment is computed per review, then aggregated per topic by counting each classified review once. A topic's sentiment is therefore the mix of its reviews, not a judgement about its label; a *single feature* topic can be mostly negative (a complaint) or mostly positive (praise). The probabilities are uncalibrated, so we report counts and shares, not averages of confidence. This is whole-review sentiment, not aspect-based sentiment: a review that praises transfers and curses the login gets one label.

A built-in check

Store reviews carry a star rating that is not an input at inference time. Where a rating exists, we compare: one and two stars should lean negative, four and five positive. Agreement there is evidence the classifier reads Spanish app reviews sensibly; disagreement shows where the two signals differ. For a store adapter trained with star-derived labels, this agreement is not an independent evaluation: use its held-out test report instead.

In [ ]:
```
refresh_sentiment_models = mo.ui.run_button(label="Refresh available models")
refresh_sentiment_models
```

Refresh available models

In [ ]:
```
from voc_demo.sentiment.model_selection import available_models
from voc_demo.settings import data_root as _selection_data_root

_ = refresh_sentiment_models.value
sentiment_model_options = {
    _domain: available_models(_selection_data_root() / "sentiment", _domain)
    for _domain in ("stores", "social")
}
stores_sentiment_model = mo.ui.dropdown(options=list(sentiment_model_options["stores"]),
value="Original RoBERTuito", label="Stores model · App Store / Google Play")
social_sentiment_model = mo.ui.dropdown(options=list(sentiment_model_options["social"]),
value="Original RoBERTuito", label="Social model · X / YouTube")
mo.vstack([
    mo.md("### Choose sentiment models\nSelect an existing checkpoint for each domain.
Local models appear after training completes; press **Refresh available models** to update
the list. Hugging Face versions require access to the private repositories through the
environment's Hugging Face login. Training and evaluation are in
`workflow04_transfer_learning.py`."),
    stores_sentiment_model, social_sentiment_model,
])
```

Choose sentiment models

Select an existing checkpoint for each domain. Local models appear after training completes; press **Refresh available models** to update the list. Hugging Face versions require access to the private repositories through the environment's Hugging Face login. Training and evaluation are in **`workflow04_transfer_learning.py`**.

Stores model · App Store / Google Play Local · 20261018T074032Z-831777f6

Social model · X / YouTube Hugging Face · v1

In [ ]:
```
from huggingface_hub.errors import HfHubHTTPError as _HubError

from voc_demo.sentiment.model_selection import classify_selected_models, resolve_models
from voc_demo.sentiment.robertuito import MAX_LENGTH as SENTIMENT_MAX_LENGTH
from voc_demo.settings import model_root as _sentiment_model_root

SENTIMENT_LABELS = ("negative", "neutral", "positive")
_selected = {
    "stores": sentiment_model_options["stores"][stores_sentiment_model.value],
    "social": sentiment_model_options["social"][social_sentiment_model.value],
}
```

```
try:
    _resolved, SENTIMENT_REVISION = resolve_models(_selected)
except (OSError, ValueError, _HubError) as _error:
    mo.stop(True, mo.callout(f"Cannot access the selected model. Check the checkpoint and
Hugging Face login. {_error}", kind="warn"))
SENTIMENT_MODEL = f"Stores: {stores_sentiment_model.value} · Social:
{social_sentiment_model.value}"
_sentiment_path = SAMPLE_DIR / "sentiment.parquet"
_sentiment_manifest = SAMPLE_DIR / "sentiment.json"
_expected_sentiment = {
    "inputs_sha256": hashlib.sha256(prepared_sample[["record_id", "input_hash",
"platform"]].to_json(orient="records").encode("utf-8")).hexdigest(),
    "model": SENTIMENT_MODEL, "revision": SENTIMENT_REVISION, "max_length":
SENTIMENT_MAX_LENGTH,
    "preprocessing": "pysentimiento.preprocess_tweet(lang='es')", "label_policy": "argmax;
ties negative, neutral, positive",
}
_saved_sentiment = json.loads(_sentiment_manifest.read_text(encoding="utf-8")) if
_sentiment_manifest.exists() else None
if _saved_sentiment == _expected_sentiment and _sentiment_path.exists():
    sentiment = pd.read_parquet(_sentiment_path)
    _origin = f"loaded from `{_sentiment_path.name}`"
else:
    _t0 = time.perf_counter()
    with mo.status.spinner(title="Classifying reviews with the selected models…"):
        _predictions, _diagnostics = classify_selected_models(
            prepared_sample[["record_id", "embedding_text", "platform"]], _resolved,
            _sentiment_model_root() / "sentiment",
            sentiment_run_id="prototypeV0-workflow01",
dataset_release_id=snapshot.info["release_id"],
        )
    _truncated = {d["record_id"]: bool(d["truncated"]) for d in _diagnostics}
    sentiment = pd.DataFrame(
        [
            {
                "record_id": p["record_id"], "status": p["status"], "label": p["label"],
                **{f"p_{k}": (p["probabilities"] or {}).get(k) for k in SENTIMENT_LABELS},
                "truncated": _truncated.get(p["record_id"], False),
            }
            for p in _predictions
        ]
    )
    sentiment.to_parquet(_sentiment_path, index=False)
    _sentiment_manifest.write_text(json.dumps(_expected_sentiment, indent=2),
encoding="utf-8")
    _origin = f"classified now in {time.perf_counter() - _t0:.0f} s and saved to
`{_sentiment_path.name}`"
assert list(sentiment["record_id"]) == list(prepared_sample["record_id"])
_counts = sentiment["label"].value_counts()
mo.md(
    f"**{int(sentiment['status'].eq('classified').sum()):,}** reviews classified with
`{SENTIMENT_MODEL}` · "
    + " · ".join(f"{k} **{int(_counts.get(k, 0))}** ({_counts.get(k, 0) /
len(sentiment):.0%})" for k in SENTIMENT_LABELS)
    + f" · {int(sentiment['truncated'].sum())} truncated at {SENTIMENT_MAX_LENGTH} tokens ·
```

```
{_origin}"
)
```

```
Loading weights:   0%|          | 0/201 [00:00<?, ?it/s]
Loading weights: 100%|██████████| 201/201 [00:00<00:00, 4599.75it/s]

Loading weights:   0%|          | 0/201 [00:00<?, ?it/s]
Loading weights: 100%|██████████| 201/201 [00:00<00:00, 4493.06it/s]
```

1,000 reviews classified with `Stores: Local · 20261018T074032Z-831777f6 · Social: Original RoBERTuito` · negative 279 (28%) · neutral 55 (6%) · positive 666 (67%) · 3 truncated at 128 tokens · classified now in 38 s and saved to `sentiment.parquet`

In [ ]:

```
# One row per topic: label counts and shares, joined with the model's label and
assessment.
reviewed = clustered.assign(topic=topic_ids).join(sentiment.set_index("record_id")
[["label", "p_negative", "p_neutral", "p_positive"]], on="record_id")
_counts = pd.crosstab(reviewed["topic"],
reviewed["label"]).reindex(columns=SENTIMENT_LABELS, fill_value=0)
sentiment_by_topic = (
    _counts.assign(reviews=_counts.sum(axis=1))
    .assign(**{f"{k}_share": lambda d, k=k: d[k] / d["reviews"] for k in
SENTIMENT_LABELS})
    .join(topic_labels.set_index("topic")[["label", "assessment"]].rename(columns=
{"label": "feature"}), how="left")
    .reset_index()
    .sort_values(["negative_share", "reviews"], ascending=[False, False])
    .reset_index(drop=True)
)
sentiment_by_topic["feature"] = sentiment_by_topic["feature"].fillna("—")
sentiment_by_topic.loc[sentiment_by_topic["topic"] == -1, ["feature", "assessment"]] = ["
(unassigned)", "—"]
_overall = reviewed["label"].value_counts(normalize=True)
mo.vstack(
    [
        mo.md(
            f"Overall: negative **{_overall.get('negative', 0):.0%}**, neutral **
{_overall.get('neutral', 0):.0%}**, "
            f"positive **{_overall.get('positive', 0):.0%}**. Per topic, sorted by
negative share:"
        ),
        mo.ui.table(
            sentiment_by_topic[["topic", "reviews", "negative", "neutral", "positive",
"negative_share", "positive_share", "feature", "assessment"]]
            .round({"negative_share": 3, "positive_share": 3}),
            label="Sentiment mix per topic (each classified review counts once)",
page_size=10,
        ),
    ]
)
```

Overall: negative **28%**, neutral **6%**, positive **67%**. Per topic, sorted by negative share:

Sentiment mix per topic (each classified review counts once)

Search...

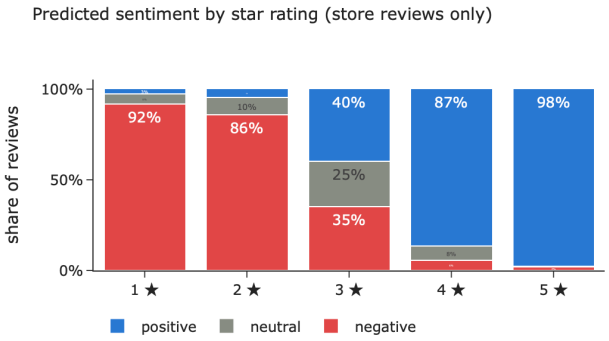
topic int64	reviews int64	negative int64	neutral int64	positive int64	negative_share float64	positive_share float64	feature object unique: 12
25	13	13	0	0	1.000	0.000	–
33	10	9	0	1	0.900	0.100	–
19	15	13	2	0	0.867	0.000	Gestión de pag
22	14	12	2	0	0.857	0.000	–
6	34	27	6	1	0.794	0.029	Mejorar la esta
14	21	16	5	0	0.762	0.000	–
0	96	68	13	15	0.708	0.156	Mejorar la esta
34	10	7	0	3	0.700	0.300	Gestión de tar
15	19	13	2	4	0.684	0.211	–
27	12	8	0	4	0.667	0.333	Visualización

38 rows, 9 columns *No selection*

In [ ]:
```
# Store reviews carry a star rating the classifier never saw: compare the two signals.
_rated = reviewed.join(prepared_sample["rating"], rsuffix="_r")
_rated = _rated[_rated["rating"].notna()].copy()
_rated["rating"] = _rated["rating"].astype(int)
rating_vs_sentiment = pd.crosstab(_rated["rating"],
_rated["label"]).reindex(columns=SENTIMENT_LABELS, fill_value=0)
_expected = _rated["rating"].map(lambda r: "negative" if r <= 2 else "neutral" if r == 3
else "positive")
_agree = (_expected == _rated["label"]).mean()
_strict = _rated[_rated["rating"] != 3]
_agree_strict = ((_strict["rating"] <= 2) == (_strict["label"] ==
"negative")).where(_strict["label"] != "neutral").dropna().mean()
_odd = _rated[(_rated['rating'] == 5) & (_rated['label'] == 'neutral')]
_share = rating_vs_sentiment.div(rating_vs_sentiment.sum(axis=1), axis=0)
_figure = go.Figure()
for _k, _c in zip(SENTIMENT_LABELS, ("#e34948", "#8a8d85", "#2a78d6")):
    _figure.add_bar(name=_k, x=[f"{r} ★" for r in _share.index], y=_share[_k], marker=
{"color": _c}, text=[f"{v:.0%}" for v in _share[_k]], textposition="inside")
_figure.update_layout(barmode="stack", height=340, template="simple_white", title={"text":
"Predicted sentiment by star rating (store reviews only)", "font": {"size": 14}}, yaxis=
{"tickformat": ".0%", "title": "share of reviews"}, legend={"orientation": "h", "y":
-0.2})
mo.vstack(
    [
        mo.md(
            f"**{len(_rated):,}** reviews have a rating. Mapping 1-2 ★ → negative, 3 ★
→ neutral, 4-5 ★ → positive, the classifier "
            f"agrees with the rating on **{_agree:.0%}** of them; ignoring the neutral
middle on both sides, "
            f"polarity agrees on **{_agree_strict:.0%}**. The main disagreement: **
{len(_odd)}** five-star reviews predicted "
            f"neutral, median {int(_odd['embedding_text'].str.len().median())} characters,
typically one-word praise such as "
            f"“{'”, “'.join(_odd['embedding_text'].str.slice(0, 20).head(3))}”: the
classifier, trained on tweets, keeps very short "
            f"praise in neutral unless it carries an explicit cue."
```

```
        ),
        mo.hstack([mo.ui.plotly(_figure), mo.ui.table(rating_vs_sentiment.reset_index(),
label="Counts: rating × predicted label")], widths=[3, 2]),
    ]
)
```

904 reviews have a rating. Mapping 1–2 ★ → negative, 3 ★ → neutral, 4–5 ★ → positive, the classifier agrees with the rating on **90%** of them; ignoring the neutral middle on both sides, polarity agrees on **97%**. The main disagreement: **2** five-star reviews predicted neutral, median 102 characters, typically one-word praise such as “La aplicación era mu”, “no recibo Mensajes c”: the classifier, trained on tweets, keeps very short praise in neutral unless it carries an explicit cue.



Counts: rating × predicted label

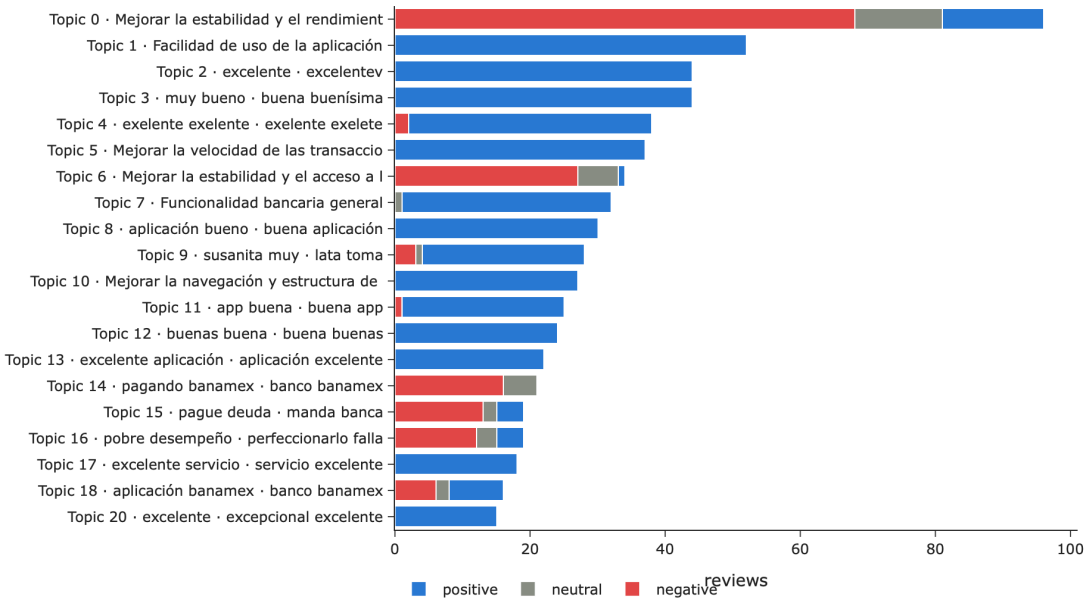
Search...

rating int64	negative int64	neutral int64	positi int64
1	130	8	
2	36	4	
3	21	15	
4	5	7	
5	10	2	

5 rows, 4 columns

In [ ]:
```
_plot = sentiment_by_topic[sentiment_by_topic["topic"] >= 0].sort_values("reviews",
ascending=True).tail(20)
_names = [f"Topic {t} · {('' if f == '–' else f)[:38] or k}" for t, f, k in
zip(_plot["topic"], _plot["feature"], [" · ".join(w for w, _ in topic_aspects["mmr_0.3"]
[int(t)][:2]) for t in _plot["topic"]])]
_figure = go.Figure()
for _k, _c in zip(SENTIMENT_LABELS, ("#e34948", "#8a8d85", "#2a78d6")):
    _figure.add_bar(name=_k, y=_names, x=_plot[_k], orientation="h", marker={"color": _c})
_figure.update_layout(barmode="stack", height=620, template="simple_white", margin={"l":
300}, title={"text": "The twenty largest topics, reviews by predicted sentiment", "font":
{"size": 14}}, xaxis_title="reviews", legend={"orientation": "h", "y": -0.08})
mo.ui.plotly(_figure)
```

The twenty largest topics, reviews by predicted sentiment



In [ ]:
```
_options = {
    f"Topic {int(r.topic)} · {int(r.reviews)} reviews · {r.negative_share:.0%} negative" +
("" if r.feature in ("-", "(unassigned)") else f" · {r.feature}"): int(r.topic)
    for r in sentiment_by_topic.sort_values("reviews", ascending=False).itertuples()
}
sentiment_choice = mo.ui.dropdown(options=_options, value=next(iter(_options)),
label="Read the reviews of")
sentiment_choice
```

Read the reviews of Topic -1 · 155 reviews · 38% negative

In [ ]:
```
_topic = int(sentiment_choice.value)
_members = reviewed[reviewed["topic"] == _topic].copy()
_members["confidence"] = _members[["p_negative", "p_neutral", "p_positive"]].max(axis=1)
_members = _members.sort_values(["label", "confidence"], ascending=[True, False])
_mix = _members["label"].value_counts()
mo.vstack(
    [
        mo.md(
            f"**Topic {_topic}** · {len(_members)} reviews · "
            + " · ".join(f"{k} {int(_mix.get(k, 0))}" for k in SENTIMENT_LABELS)
            + ". Sorted by label, most confident first; `confidence` is the winning
probability (uncalibrated)."
        ),
        mo.ui.table(
            _members[["record_id", "platform", "label", "confidence", "p_negative",
"p_neutral", "p_positive", "embedding_text"]].round(3),
            label="Reviews with their predicted sentiment", page_size=10,
        ),
    ]
)
```

Topic -1 · 155 reviews · negative 59 · neutral 16 · positive 80. Sorted by label, most confident first; `confidence` is the winning probability (uncalibrated).

Reviews with their predicted sentiment

Search...

	Index0 int64	record_id object unique: 155	platform string unique: 4	label object unique: 3	confidence float64 0.45 1	p_negative float64 0 0.98	p_neutral float64 0
☐	**880**	youtube:-:UgxuwZNqztWoBapLfF54AaABAg	youtube	negative	0.983	0.983	
☐	**272**	play:com.citibanamex.banamexmobile:d1cd0b...	google_play	negative	0.982	0.982	
☐	**976**	play:com.citibanamex.banamexmobile:5aa7d7...	google_play	negative	0.982	0.982	
☐	**281**	play:com.citibanamex.banamexmobile:291f5af...	google_play	negative	0.978	0.978	
☐	**322**	play:com.citibanamex.banamexmobile:8d9d29...	google_play	negative	0.977	0.977	
☐	**241**	appbot:498483038:3752339638	app_store	negative	0.975	0.975	
☐	**400**	play:com.citibanamex.banamexmobile:9f2157...	google_play	negative	0.974	0.974	
☐	**909**	play:com.citibanamex.banamexmobile:efee8b...	google_play	negative	0.974	0.974	
☐	**462**	play:com.citibanamex.banamexmobile:32285b...	google_play	negative	0.971	0.971	
☐	**968**	play:com.citibanamex.banamexmobile:d9f776...	google_play	negative	0.970	0.970	

155 rows, 8 columns *No selection*

GENERAL CONCLUSIONS

The project demostrates the factability of analyzing customer reviews for App Store, Google Play, X, and Youtube to create recurring topics and sentiment summaries, that support the identification of customer concerns. The pipeline combines semantic embeddings, dimensionality reduction, clustering, topic interpretation and supervised sentiment clasification.

What algorithms can serve as baselines for the project?

Review grouping

HBDSCAN, KMEANS

Topic Representation

FT-IDF, BERTopic

Sentiment clasification

RoBERTuito, modified-RoBERTuito (Transfer learning)

Can feature importance be determined?

It depends on the component, BERTopic's representative words and reviews help explain what characterized a determined topic, however the do not establish the features that cause a sentient prediction.

Is the project underfiting or overtifing

There is no single conclusion for the entire pipeline, topic modelling needs a check for excessive fragmentation, broad groups, outliers and stability, and in the sentiment analysis part both models didn't show strong evidence of overfitting.

Whats metrics are appropiate for the bussines problem?

Evaluation should combine

Topics: human assessment of coherence and relevance

Clustering: silhouette, cluster stability

Sentiment: f1, presicion, recall

Whats minimum performance should be achieved?

Topics should demostrate coherent, relevant groups, based on human-review criteria, a sentiment target is f1 ≥ 0.65 and per-class F1 ≥ 0.50

In [ ]:
```
# Projection for the Studio "Sentiment" page: a label and three probabilities per row, the per-topic mix joined
# with the labels, the rating check, and the model identity.
sentiment_payload = {
    "model": {"id": SENTIMENT_MODEL, "revision": SENTIMENT_REVISION, "max_length": SENTIMENT_MAX_LENGTH, "labels": list(SENTIMENT_LABELS), "preprocessing": "pysentimiento Spanish preprocessing"},
    "counts": {"rows": int(len(sentiment)), "classified": int(sentiment["status"].eq("classified").sum()), "truncated": int(sentiment["truncated"].sum()),
               **{k: int((sentiment["label"] == k).sum()) for k in SENTIMENT_LABELS}},
    "label_of_row": [str(l) for l in sentiment["label"]],
    "rating_of_row": [None if pd.isna(r) else int(r) for r in prepared_sample["rating"]],
    "probabilities": [[round(float(r.p_negative), 3), round(float(r.p_neutral), 3), round(float(r.p_positive), 3)] for r in sentiment.itertuples()],
    "topics": [
        {"topic": int(r.topic), "reviews": int(r.reviews), **{k: int(getattr(r, k)) for k in SENTIMENT_LABELS},
         "feature": None if r.feature in ("–", "(unassigned)") else r.feature, "assessment": None if r.assessment == "–" else r.assessment}
        for r in sentiment_by_topic.itertuples()
    ],
    "rating_check": {
        "rated": int(rating_vs_sentiment.to_numpy().sum()),
        "rows": [{"rating": int(i), **{k: int(v) for k, v in row.items()}} for i, row in rating_vs_sentiment.iterrows()],
    },
}
mo.md(f"`sentiment_payload` ready for the Studio *Sentiment* page · {len(json.dumps(sentiment_payload, ensure_ascii=False).encode()) / 1000:,.0f} kB")
```

`sentiment_payload` ready for the Studio *Sentiment* page · 44 kB

In [ ]:
```
# Projection for the Studio "Dashboard" page: the store catalogue (every cached artifact of this prototype,
# with the manifest that keys it) and the month of each review, so the dashboard can filter by time.
# All analytical values come from the earlier payloads; this cell adds no computation.
_describe = {
    "sample.parquet": "the 1,000 sampled reviews (workable rows)",
    "embeddings.npy": "384-d vectors, one row per review",
    "embeddings.json": "manifest: model, revision, inputs fingerprint",
    "reduced.npz": "UMAP 5-d (clustering) and 2-d (display) coordinates",
    "reduced.json": "manifest: UMAP parameters, vectors fingerprint",
    "clusters.npz": "HDBSCAN labels and membership probabilities",
    "clusters.json": "manifest: HDBSCAN parameters, reduction fingerprint",
    "representations.json": "keywords per topic: c-TF-IDF, KeyBERT, KeyBERT → MMR",
    "labels.json": "one answer per topic from the language model, keyed by evidence",
    "labels.uuid-ids.json": "earlier answers (full record ids), kept aside, unused",
    "sentiment.parquet": "label and three probabilities per review",
    "sentiment.json": "manifest: classifier, revision, inputs fingerprint",
}
```

```
def _manifest_summary(path):
    try:
        data = json.loads(path.read_text(encoding="utf-8"))
    except (OSError, ValueError):
        return None
    if isinstance(data, dict) and "manifest" in data:
        data = data["manifest"]
    if isinstance(data, dict) and "attempts" in data:
        return {"model": data.get("model"), "prompt_version": data.get("prompt_version"),
"answers": len(data.get("attempts", {}))}
    return {k: v for k, v in data.items() if isinstance(v, (str, int, float, bool)) and
not str(k).endswith("sha256")} if isinstance(data, dict) else None


dashboard_payload = {
    "store": {
        "root": str(SAMPLE_DIR),
        "files": [
            {
                "name": p.name, "bytes": p.stat().st_size, "role": _describe.get(p.name,
"artifact"),
                "modified": pd.Timestamp(p.stat().st_mtime, unit="s",
tz="UTC").isoformat(timespec="seconds"),
                "manifest": _manifest_summary(p) if p.suffix == ".json" else None,
            }
            for p in sorted(SAMPLE_DIR.iterdir()) if p.is_file()
        ],
    },
    "release": {"release_id": snapshot.info["release_id"], "created_at":
snapshot.info["created_at"], "schema_version": snapshot.info["schema_version"]},
    "month_of_row": [str(d)[:7] for d in prepared_sample["record_date"]],
}
mo.md(f"`dashboard_payload` ready for the Studio *Dashboard* page ·
{len(dashboard_payload['store']['files'])} stored artifacts, {sum(f['bytes'] for f in
dashboard_payload['store']['files']) / 1e6:.1f} MB")
```

`dashboard_payload` ready for the Studio *Dashboard* page · 12 stored artifacts, 2.1 MB

S3 demo checkpoint

This demo loads its pinned snapshot from `voc/demos/prototypeV0` and keeps a working copy under `.data/workflow01/<snapshot-id>/`. The repository does not include a snapshot fallback. If S3 cannot provide a verified checkpoint, loading stops before the processing stages.

Existing complete caches are preserved. To verify a fresh download, use an empty runtime directory as described in the demo README. To share changed results, finish every stage and press Publish demo to S3. This does not change the shared dataset. Record the new ID before updating the demo pin.

In [ ]:
```
publish_demo = mo.ui.run_button(label="Publish demo to S3 · share the complete current checkpoint")
publish_demo
```

Publish demo to S3 · share the complete current checkpoint

In [ ]:
```
from voc_demo.checkpoints import publish_prototype_demo

mo.stop(not publish_demo.value, mo.md("Publishing is explicit. Press the button after completing the notebook."))
_id = publish_prototype_demo(
    SAMPLE_DIR, release_id=snapshot.info["release_id"], workable=workable,
)
mo.md(f"**Demo published:** `{_id}`. Teammates with an empty cache can load it from S3.")
```

```
Traceback (most recent call last):
  File "", line 4, in <module>
    _id = publish_prototype_demo(
          ^^^^^^^^^^^^^^^^^^^^^^^
  File "/workspace/notebooks/demos/voc_demo/checkpoints.py", line 195, in publish_prototype_demo
    keys = validate_checkpoint(frozen, workable, release_id)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/workspace/notebooks/demos/voc_demo/checkpoints.py", line 67, in validate_checkpoint
    raise ValueError("Stale stage input fingerprint: " + key)
ValueError: Stale stage input fingerprint: sentiment.json
```

Published stores v3 transfer-learning report ready for the Studio Sentiment page.